EduGenie: Google Gemini Powered Learning Assistant

Complete Project Documentation – 8 Phases + Five Input Area Screenshots

This consolidated document contains the complete project details for all eight required phases. The 15 supplied application screenshots are organized under the five functional input areas: Q&A, Explain a Concept, Generate a Quiz, Summarize Text, and Recommend a Learning Path.

EduGenie: Google Gemini Powered Learning Assistant

COMPLETE PROJECT DOCUMENTATION — PHASES 1 TO 8

Prepared from the supplied project document. Replace all square-bracket placeholders with actual team details, implementation evidence, test results, and submission links.

PHASE 1: Brainstorming & Ideation

1.1 Background and context

Students often need quick explanations, clarification of doubts, revision material, practice questions, and guidance on what to study next. Learning resources may be lengthy or written at a level that is difficult for a beginner.

EduGenie proposes a single lightweight web interface through which learners can access several AI-supported study tasks. The project document positions the application for students across academic levels and self-learners.

1.2 Problem statement

Learners may spend considerable time locating suitable explanations, condensing long passages, preparing practice questions, and organizing a learning sequence. A simple, integrated assistant can bring these activities together and make study support easier to access.

Problem statement: How might we provide learners with a simple web-based assistant that can answer academic questions, explain concepts in accessible language, create quizzes from supplied text, summarize passages, and suggest a structured learning path?

1.3 Proposed solution

Develop EduGenie as a browser-based educational assistant with a FastAPI backend and HTML/CSS interface. The user selects a task, enters a question, topic, or passage, submits it, and views the generated response on the page.

The solution is modular: separate backend logic is planned for explanation, Q&A, quiz generation, summarization, and learning recommendations.

1.4 Brainstormed features

Question answering: respond to general-knowledge and academic questions.

Concept explanation: present complex topics in simpler, beginner-friendly language.

Quiz generation: create three multiple-choice questions from a passage, with four options per question and a correct answer.

Summarization: condense long educational passages while retaining key information.

Learning recommendations: organize beginner-to-advanced topics and suggest learning resources.

1.5 Intended users and use scenarios

Primary users: school and college students, independent learners, and users who need accessible explanations or revision support.

Scenario A: A student asks, “Which is the largest ocean?” and receives an answer.

Scenario B: A student studying the Pythagoras Theorem generates a quiz to check understanding.

Scenario C: A learner studying SQL requests a structured beginner-to-advanced learning path, with timelines and resource suggestions.

1.6 Objectives

Provide a single, easy-to-use interface for five learning-support tasks.

Use an appropriate AI model for each task as specified in the project document.

Keep the backend modular so individual features can be developed and maintained separately.

Display results in the web interface after a form submission.

Validate the five main user workflows through functional testing.

1.7 Initial feasibility and constraints

Technical feasibility: the source document lists Python 3.10+, FastAPI, Uvicorn, Jinja2, HTML/CSS, a Google Gemini API key, and the local LaMini-Flan-T5-783M model.

Operational feasibility: the application is intended to run locally through Uvicorn and be accessed at http://127.0.0.1:8000.

Constraints to manage: availability and cost/limits of the Gemini API; network access for cloud model calls; local compute requirements for the explanation model; model output variability; and protection of API credentials.

1.8 Initial success criteria

Each of the five tasks can be selected and submitted from the frontend.

The relevant backend endpoint invokes the corresponding module.

The response or a useful error message is shown to the user.

Quiz output contains three questions, four options per question, and an indicated correct answer, as specified in the project document.

Setup instructions allow another team member to run the application locally.

1.9 Ideation outcome

The selected idea is EduGenie, a modular AI-powered learning assistant. The next phase converts this concept into functional, non-functional, technical, and user requirements.

PHASE 2: Requirement Analysis

2.1 Scope

The project scope covers a web interface, a FastAPI service, five learning modules, model/API integration, local execution, and functional verification of the five user tasks. Production hosting, authentication, persistent learner profiles, analytics, mobile apps, and the future enhancements listed in the source document are outside the confirmed baseline scope.

2.2 Stakeholders and user needs

Student / learner: submit a question, topic, or passage and receive a useful learning response.

Developer / maintainer: install dependencies, configure credentials, run the service, diagnose errors, and update modules.

Faculty mentor / evaluator: review project phases, inspect the repository, and observe the demonstration.

2.3 Functional requirements

FR-01: The frontend shall provide a task selector with Explain, QnA, Quiz, Summary, and Recommend Path options.

FR-02: The frontend shall provide a text area for user input and a submit action.

FR-03: The Q&A feature shall accept a question and return an AI-generated answer using Gemini 1.5 Pro via API.

FR-04: The explanation feature shall accept a concept and return a simplified explanation using LaMini-Flan-T5-783M locally.

FR-05: The quiz feature shall generate three MCQs from a supplied passage, each with four options and a correct answer; the documented flow expects JSON output and cleans Markdown code fences before parsing.

FR-06: The summarization feature shall condense a supplied passage while retaining its core information.

FR-07: The learning-path feature shall return a structured beginner-to-advanced plan and useful resources, adapting to the learner's level where input is provided.

FR-08: The backend shall expose the documented endpoints: /qa, /explain, /quiz, /summarize, and /learn/recommendations.

FR-09: Each endpoint shall connect to its respective module logic.

FR-10: The frontend shall send a POST request and display the returned result below the input area.

FR-11: Generation, API, and parsing errors shall be handled and presented as useful messages.

2.4 Non-functional requirements

Usability: task choices, input area, submit action, and result display should be clear and responsive.

Maintainability: keep feature logic separated into modules, following the source folder architecture.

Reliability: handle missing input, model/API failures, invalid JSON, and unexpected responses without crashing the user workflow.

Performance: return results within a reasonable time for the selected model and network conditions; record actual timings during testing rather than assuming a target has been met.

Security: store the Gemini API key outside source code (for example, in an environment variable); do not commit secrets to a public repository.

Compatibility: use Python 3.10+ and test on the operating system(s) actually used by the team.

Accessibility: use readable labels, sufficient contrast, keyboard-accessible controls, and clear error messages.

2.5 Technology and environment requirements

Python 3.10 or later; FastAPI; Uvicorn; Jinja2; HTML and CSS; Google Gemini API key; and the LaMini-Flan-T5-783M model for local concept explanation.

The project document also identifies a requirements.txt file and the following module/file layout: main.py, explanation_module.py, qna.py, quiz_module.py, summary_module.py, learning_path.py, templates/index.html, and static/style.css.

Exact package versions, operating-system versions, hardware specifications, and model download procedure are not fully specified in the supplied document. Record the versions actually used in the final repository.

2.6 Interface and API requirements

Frontend: task dropdown, text input area, submit button, and result container; responsive styling is expected.

Backend endpoint mapping: /qa → Q&A; /explain → concept explanation; /quiz → quiz generation; /summarize → summary; /learn/recommendations → learning path.

Request and response field names, authentication, rate limiting, and formal API schemas are not specified in the source document. Document the actual implementation in the repository after inspecting the code.

2.7 Data, privacy, and security considerations

User inputs may contain educational text or personal information. Avoid entering sensitive personal data during demonstrations and tests.

Keep the Gemini API key in a local environment file that is excluded through .gitignore. Commit a sanitized .env.example containing variable names only, if required.

Review provider terms and data handling before using real learner data. The supplied project document notes data privacy as a development challenge but does not define a formal retention policy.

2.8 Assumptions, dependencies, and risks

Assumption: a valid Gemini API key and network connectivity are available for cloud-powered tasks.

Dependency: compatible Python packages and access to the local explanation model.

Risk: cloud API quota, connectivity, or service availability can interrupt generation. Mitigation: catch errors, explain the failure, and retest when service is restored.

Risk: generated content may be inaccurate or incomplete. Mitigation: validate educational outputs and tell users to verify important information.

Risk: model output may not be valid JSON. Mitigation: clean and parse output, validate the expected structure, and report parsing errors.

2.9 Acceptance criteria

All five functions can be reached through their corresponding UI task and backend route.

Valid input produces a visible result; invalid input and simulated service failures produce a handled error.

Quiz generation returns the documented three-question/four-option structure and a correct answer for each question.

Repository setup documentation lists the actual dependencies and run command.

Testing evidence is recorded in Phase 6.

PHASE 3: Project Design

3.1 Design overview

EduGenie follows a browser-to-backend-to-AI-module design. The HTML/CSS frontend collects a task and input. FastAPI receives the request and routes it to the corresponding module. The module calls the selected local model or Gemini API, processes the result, and returns it to the frontend.

3.2 High-level architecture

Presentation layer: templates/index.html and static/style.css.

Application/API layer: main.py using FastAPI; defines the five REST endpoints and connects them to module functions.

Feature layer: explanation_module.py, qna.py, quiz_module.py, summary_module.py, and learning_path.py.

AI layer: LaMini-Flan-T5-783M locally for concept explanation; Gemini 1.5 Pro via API for Q&A, quiz generation, summarization, and learning recommendations.

Deployment/runtime layer: Uvicorn serves the FastAPI application locally.

3.3 Architecture diagram (Mermaid)

```
flowchart TD
U[Student / Learner] --> UI[HTML + CSS Frontend]
UI --> API[FastAPI: main.py]
API --> QA[/qa/] --> QAM[qna.py] --> GEM[Gemini 1.5 Pro API]
API --> EX[/explain/] --> EXM[explanation_module.py] --> LOCAL[LaMini-Flan-T5-783M local]
API --> QUIZ[/quiz/] --> QM[quiz_module.py] --> GEM
API --> SUM[/summarize/] --> SM[summary_module.py] --> GEM
API --> LP[/learn/recommendations/] --> LPM[learning_path.py] --> GEM
GEM --> API
LOCAL --> API
API --> UI
UI --> U
```

3.4 Module design

Explanation module: receives a concept/topic, invokes LaMini-Flan-T5-783M, and returns a simplified explanation.

Q&A module: receives a question, sends it to Gemini 1.5 Pro, and returns the answer.

Quiz module: receives a passage, prompts Gemini for three MCQs with four options and correct answers, removes Markdown code fences where present, parses JSON, and handles generation/parsing errors.

Summary module: receives a long passage and asks Gemini to produce a concise version retaining core information.

Learning-path module: receives a topic and learner-level context where available; asks Gemini for beginner-to-advanced topics and resources such as videos, articles, or books.

3.5 Endpoint design

3.6 User workflow

Open the application in a browser at http://127.0.0.1:8000 after starting Uvicorn.

Choose one of the five tasks from the task dropdown.

Enter a question, topic, or passage appropriate to the selected task.

Submit the form. The frontend sends a POST request to the matching FastAPI endpoint.

The backend calls the corresponding module and AI service.

The response is returned and displayed in the result area. If an error occurs, display a helpful message.

3.7 Data and output design

The supplied document does not define a persistent database or user-account model. The documented application can therefore be designed as a request/response workflow without assuming persistent storage.

Quiz output is explicitly described as JSON. Each quiz item should represent a question, four options, and the correct answer. Exact JSON property names should match the implementation and be documented after code inspection.

Other feature outputs are described as generated text. The source does not prescribe a formal response schema for these features.

3.8 UI design specification

Place the EduGenie name and a short description at the top of the page.

Provide a clearly labelled task dropdown with Explain, QnA, Quiz, Summary, and Recommend Path.

Provide a text area with a task-appropriate placeholder and a submit button.

Display the response in a visually distinct result container below the input.

Use responsive CSS for mobile and desktop widths; provide visible focus states and readable text.

3.9 Design decisions and limitations

The source document specifies the main components and model assignments but does not provide exact wireframes, database schemas, authentication design, deployment topology, or API field names. Those details must be completed from the team's actual code and design choices.

Gemini model availability and naming may change over time. Preserve the model/version actually configured in the code and note any approved substitution in the final documentation.

PHASE 4: Project Planning

4.1 Planning approach

Organize the work into milestones: model selection and architecture; core functionality and API development; frontend development and live integration; local deployment and functional testing. These milestones follow the supplied project document.

4.2 Work breakdown structure

Work package 1 — Project setup: confirm team, repository, Python environment, dependencies, and API credential handling.

Work package 2 — Architecture and model setup: establish the folder structure; configure Gemini API access; prepare the local explanation model.

Work package 3 — Feature modules: implement explanation, Q&A, quiz, summary, and learning-path logic independently.

Work package 4 — API integration: define and connect the five FastAPI endpoints.

Work package 5 — Frontend: implement the task dropdown, input form, submit behavior, result display, and responsive CSS.

Work package 6 — Integration and testing: verify all routes, model responses, error handling, and quiz JSON parsing.

Work package 7 — Documentation and demonstration: complete the eight phase records, README, screenshots, and demo recording.

4.3 Suggested team roles

Team Lead / Integration Coordinator: tracks milestones, reviews pull requests, coordinates integration, and checks final submission.

Backend / API Developer: develops main.py, FastAPI routes, request validation, and module integration.

AI Module Developer: implements model calls, prompts, JSON handling, and error handling across AI modules.

Frontend Developer: develops templates/index.html, static/style.css, and form/result interactions.

Testing / Documentation Coordinator: maintains test cases, evidence, setup instructions, and demo script.

For teams of two to three, combine roles and record shared responsibilities. Team size should follow the programme instruction (2–5 members; standard 4).

4.4 Editable responsibility matrix

4.5 Suggested schedule

This is a suggested planning template, not a schedule supplied by the project document. Replace it with the team's actual dates and progress.

4.6 Tools and communication

Version control: Git and a public GitHub repository, as required by the submission instructions.

Development: Python 3.10+, FastAPI, Uvicorn, Jinja2, HTML/CSS, Gemini API, and the local LaMini-Flan-T5-783M model.

Coordination: agree on a communication channel, short progress check-ins, issue ownership, and a review process before merging changes.

4.7 Risk register

4.8 Progress tracking

At each checkpoint, update task status, owner, completion date, blockers, and evidence link. Do not mark a task complete until the feature has been run and its evidence has been saved.

Suggested status values: Not started; In progress; Blocked; In review; Completed.

PHASE 5: Project Development

5.1 Development environment

5.0 Five Input Areas – Application Screenshots

Input Area 1 – Q&A; / Ask a Question

The learner enters a question and receives an AI-generated answer.

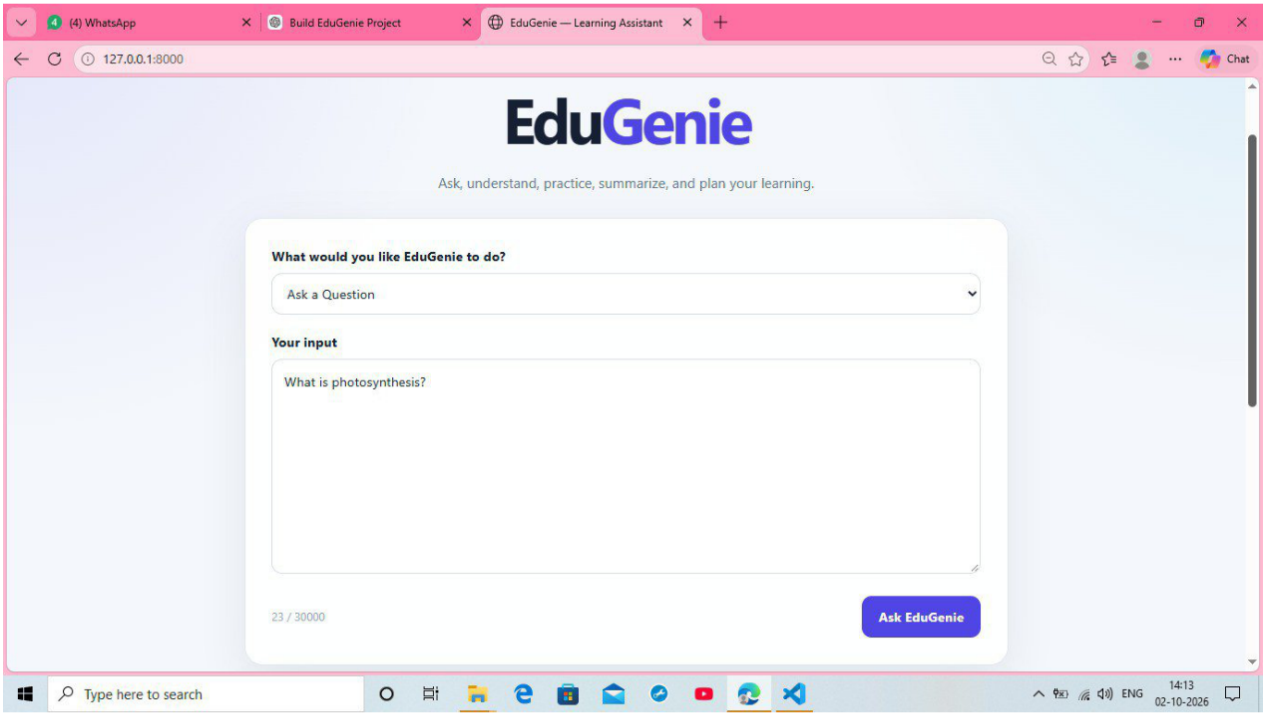


Figure 5.1: Q&A; input: a photosynthesis question entered by the learner.

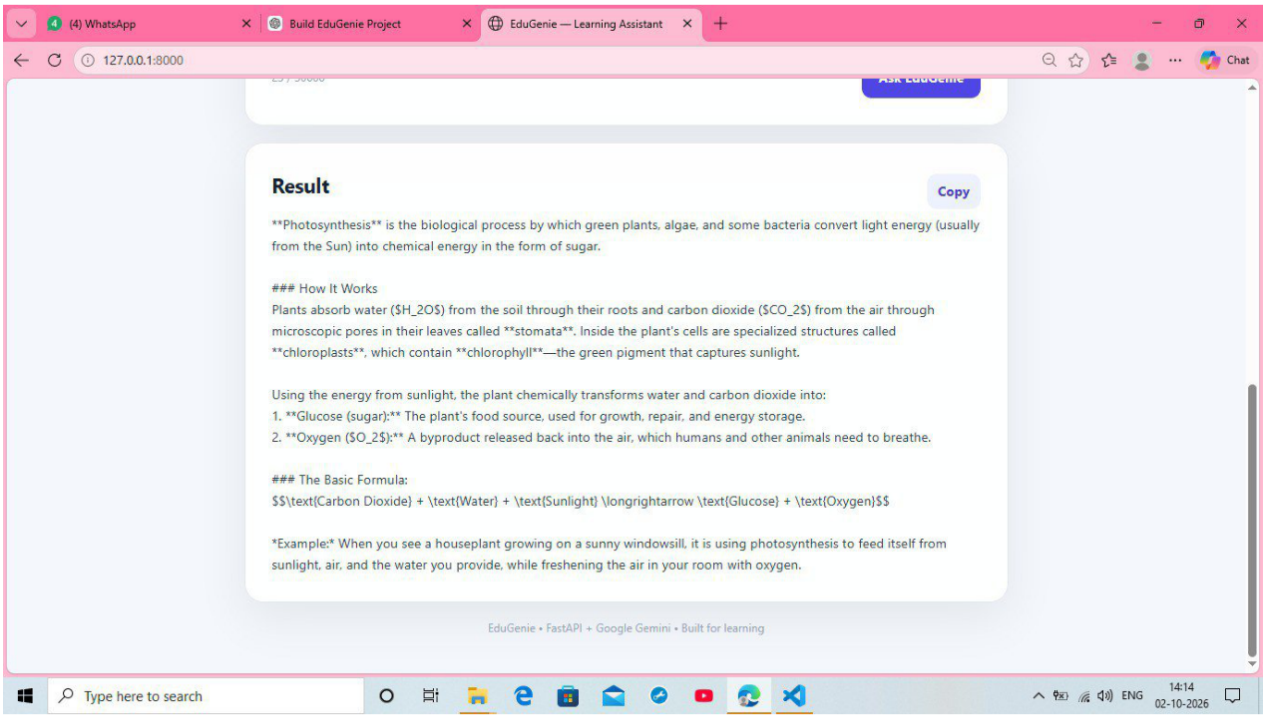


Figure 5.2: Q&A; output: generated explanation of photosynthesis.

Input Area 2 – Explain a Concept

The learner enters a concept and receives a simplified, step-by-step explanation.

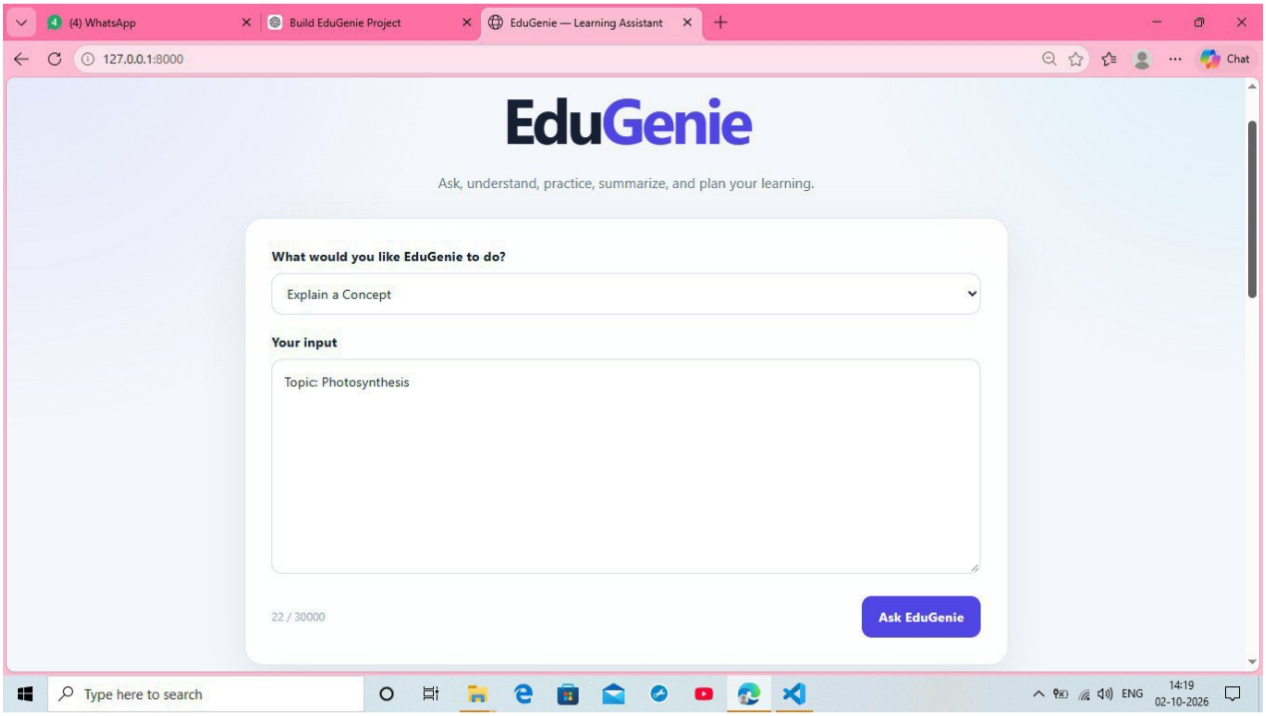


Figure 5.3: Explain a Concept input: photosynthesis entered as the topic.

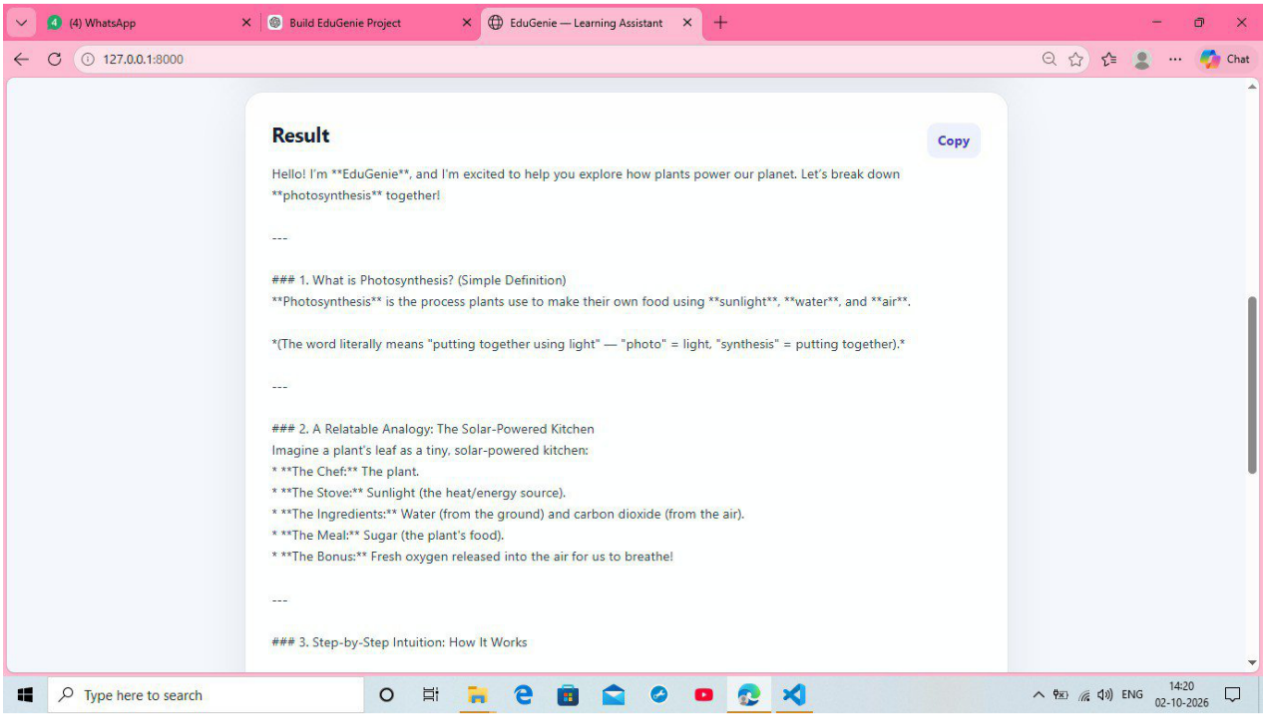


Figure 5.4: Explain a Concept output: simplified explanation with analogy and key points.

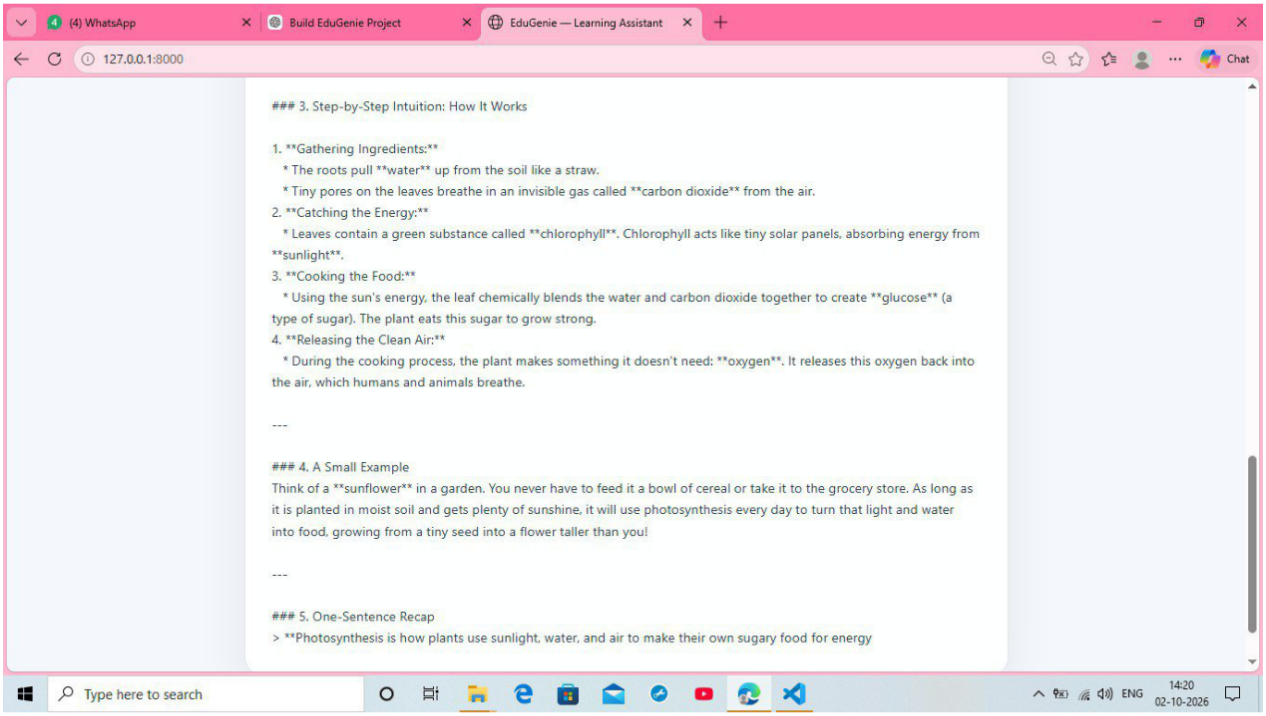


Figure 5.5: Explain a Concept output: continued step-by-step explanation and recap.

Input Area 3 – Generate a Quiz

The learner enters a topic and receives multiple-choice quiz questions with answer choices and feedback.

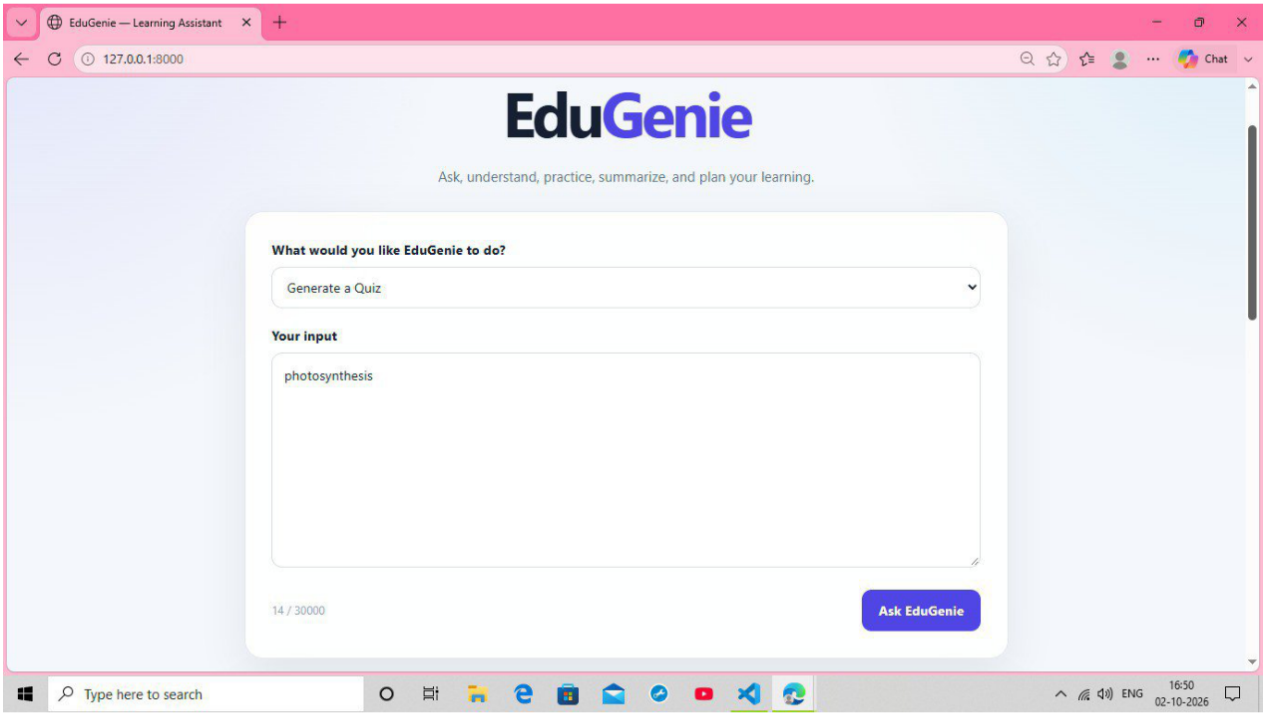


Figure 5.6: Quiz input: photosynthesis entered as the quiz topic.

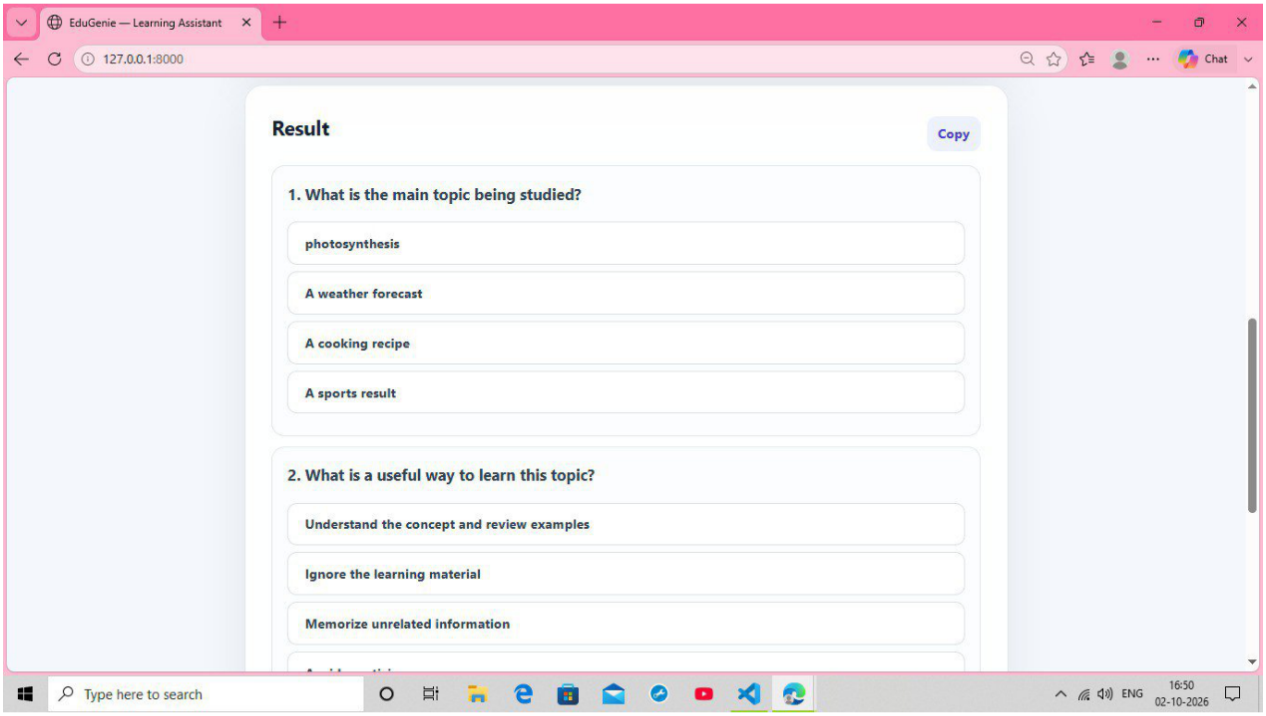


Figure 5.7: Quiz output: generated multiple-choice questions and answer options.

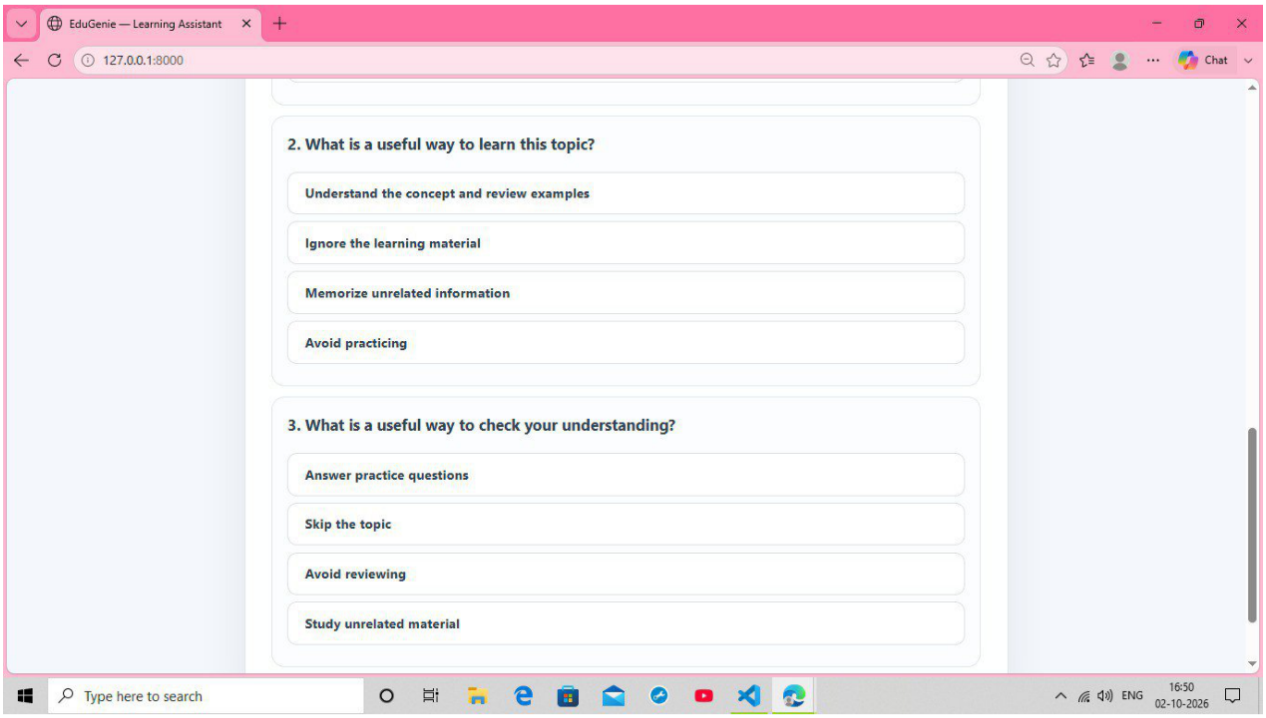


Figure 5.8: Quiz output: additional multiple-choice questions and options.

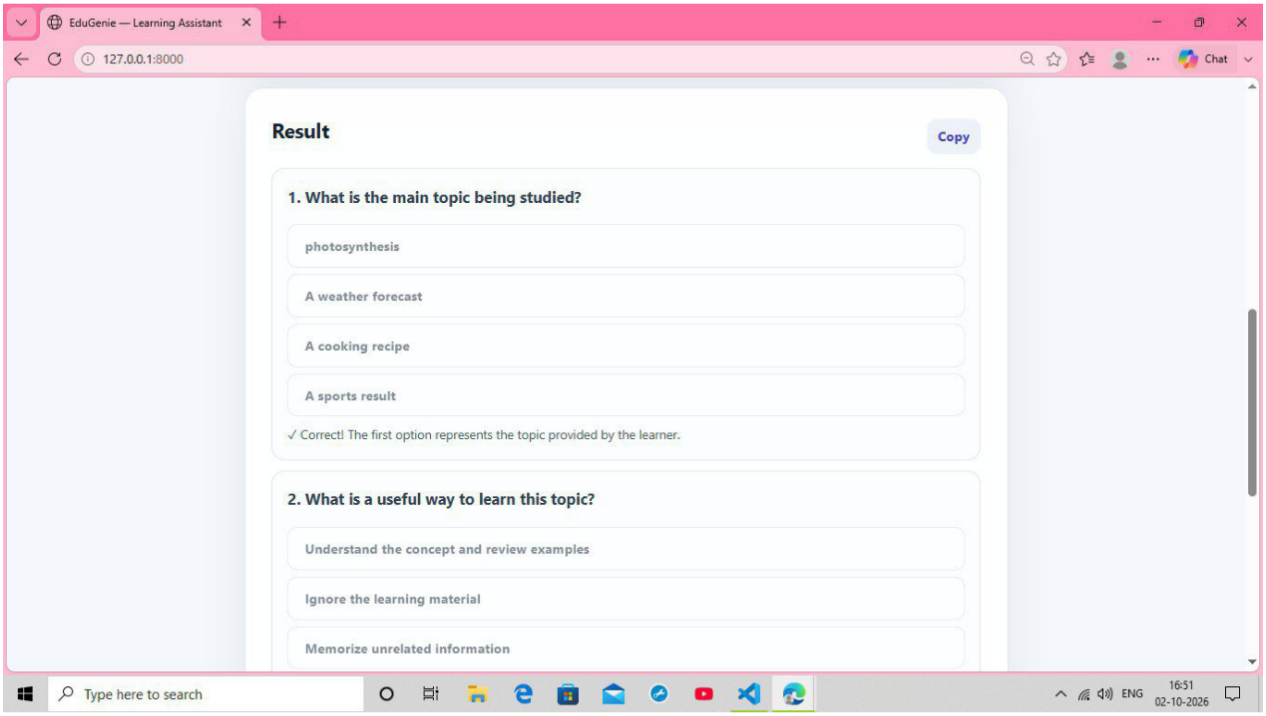


Figure 5.9: Quiz output: answer selection and correctness feedback.

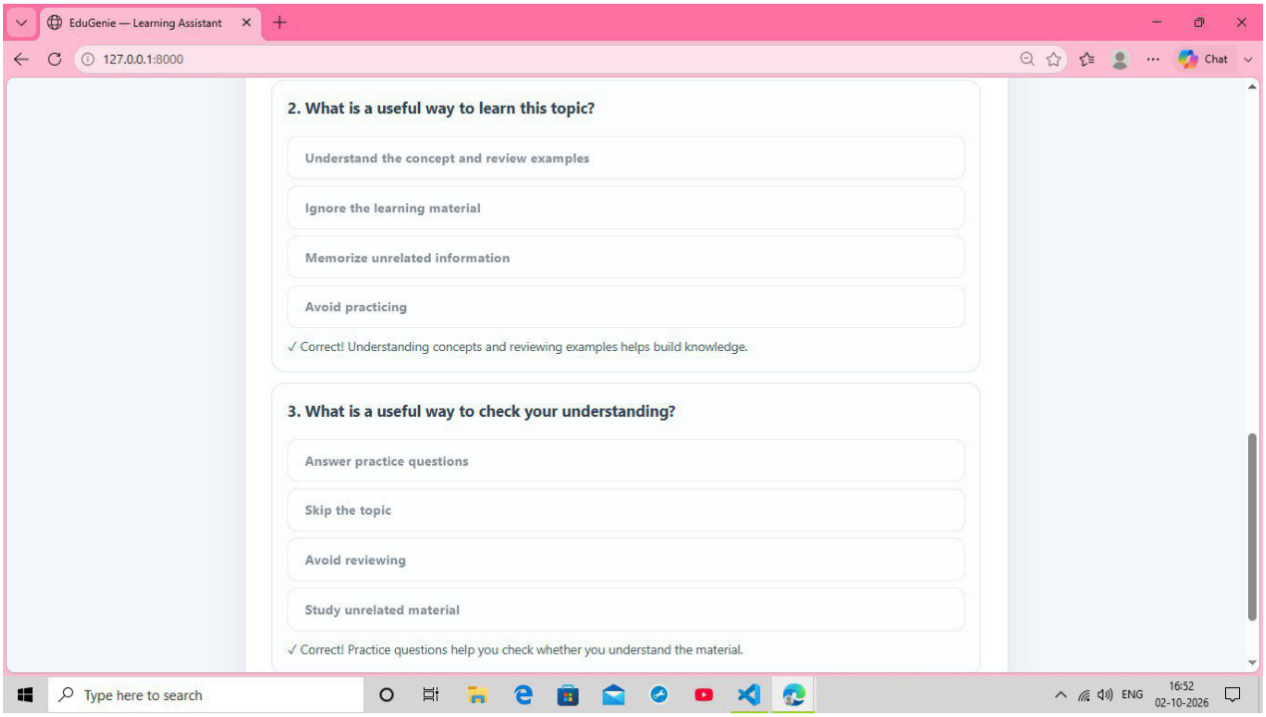


Figure 5.10: Quiz output: further answer feedback and explanations.

Input Area 4 – Summarize Text

The learner supplies educational text and receives a concise summary.

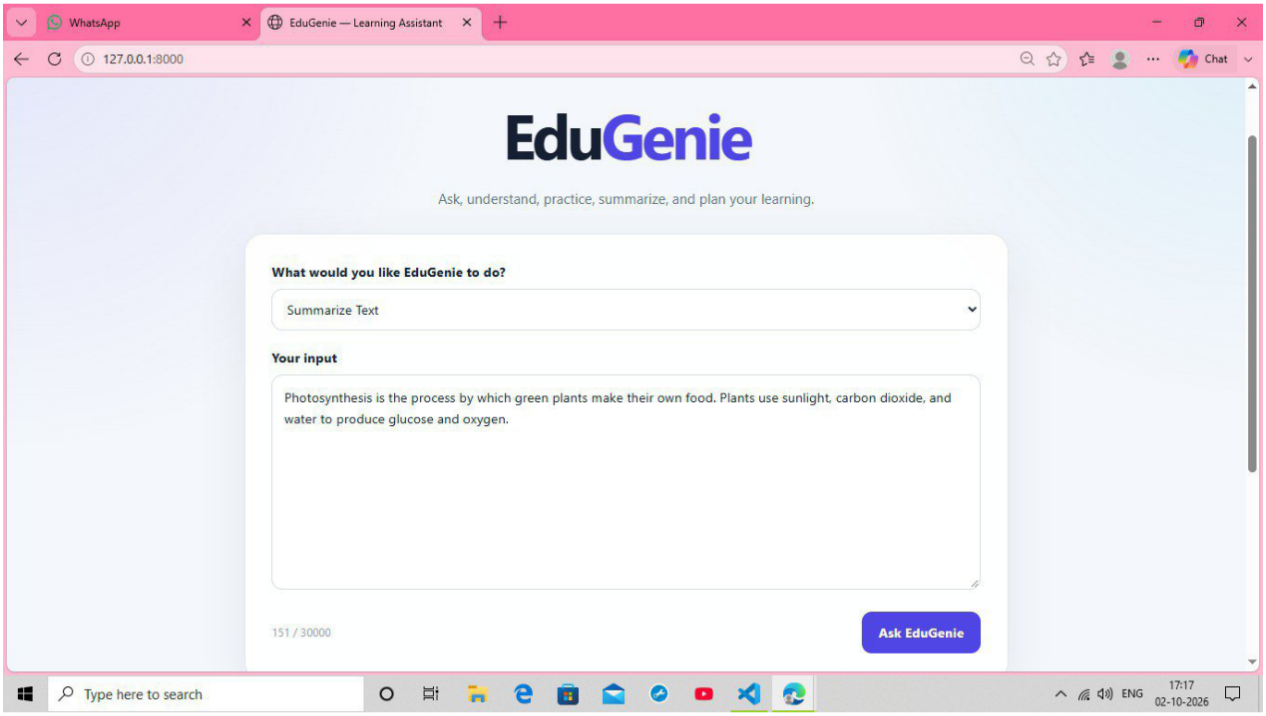


Figure 5.11: Summary input: an educational photosynthesis passage supplied for summarization.

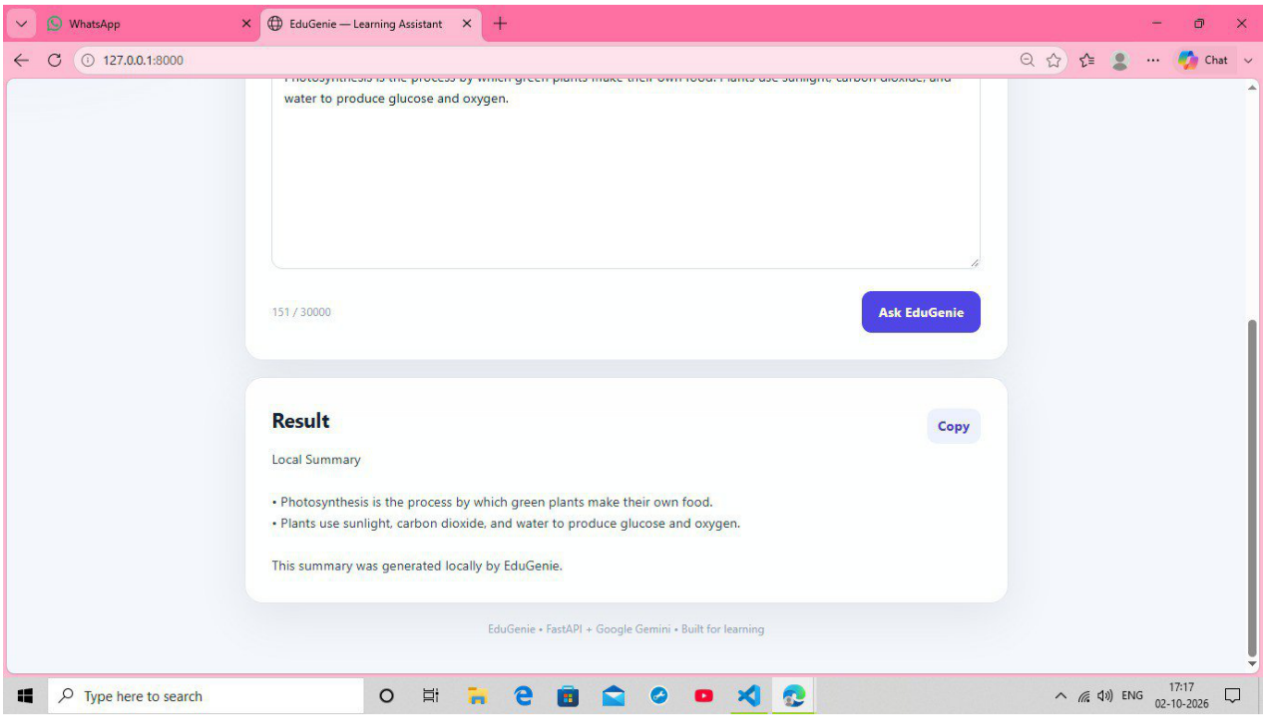


Figure 5.12: Summary output: concise summary generated from the supplied passage.

Input Area 5 – Recommend a Learning Path

The learner supplies a subject and receives a structured learning path with stages, timeline, practice and assessment guidance.

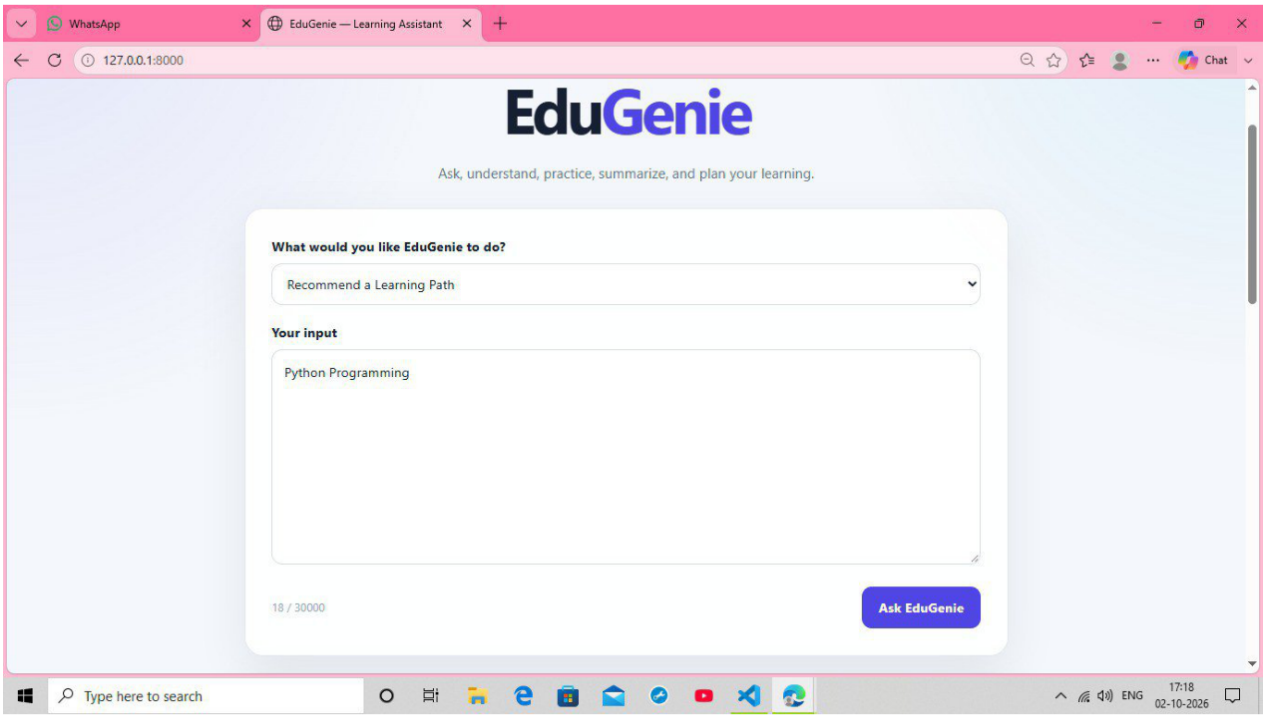


Figure 5.13: Learning Path input: Python Programming entered as the learning subject.

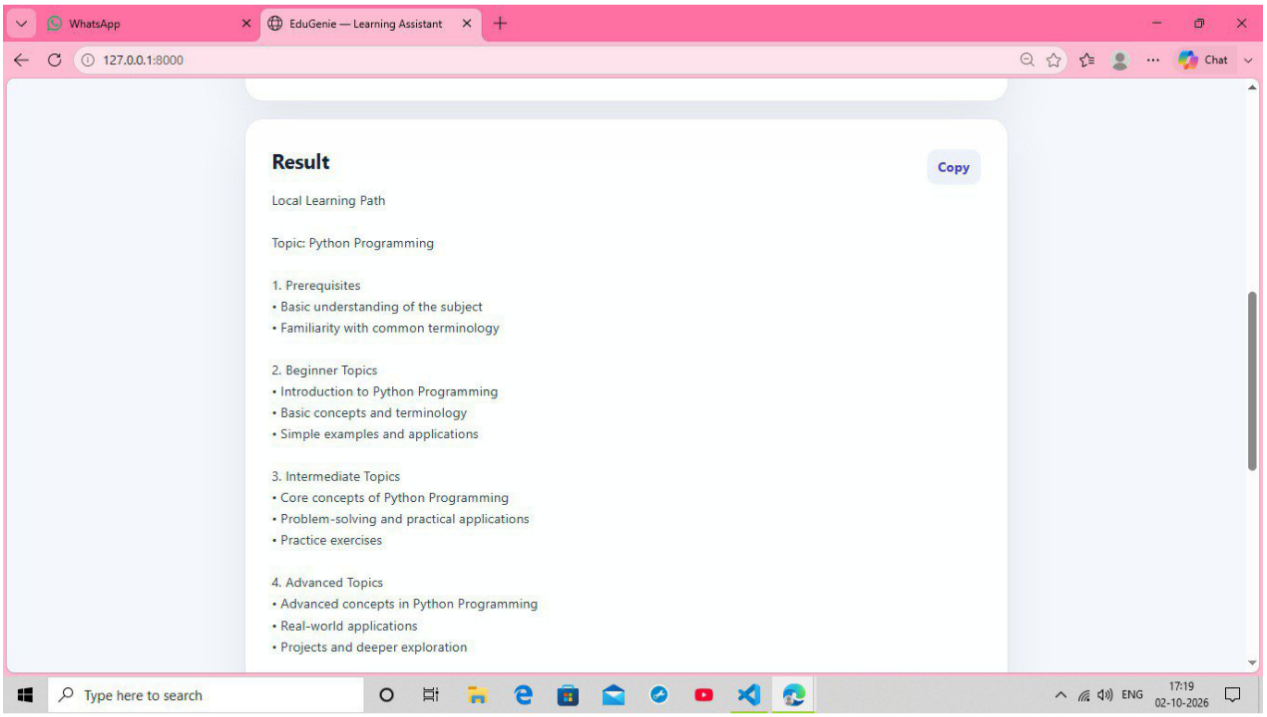


Figure 5.14: Learning Path output: prerequisite, beginner, intermediate and advanced stages.

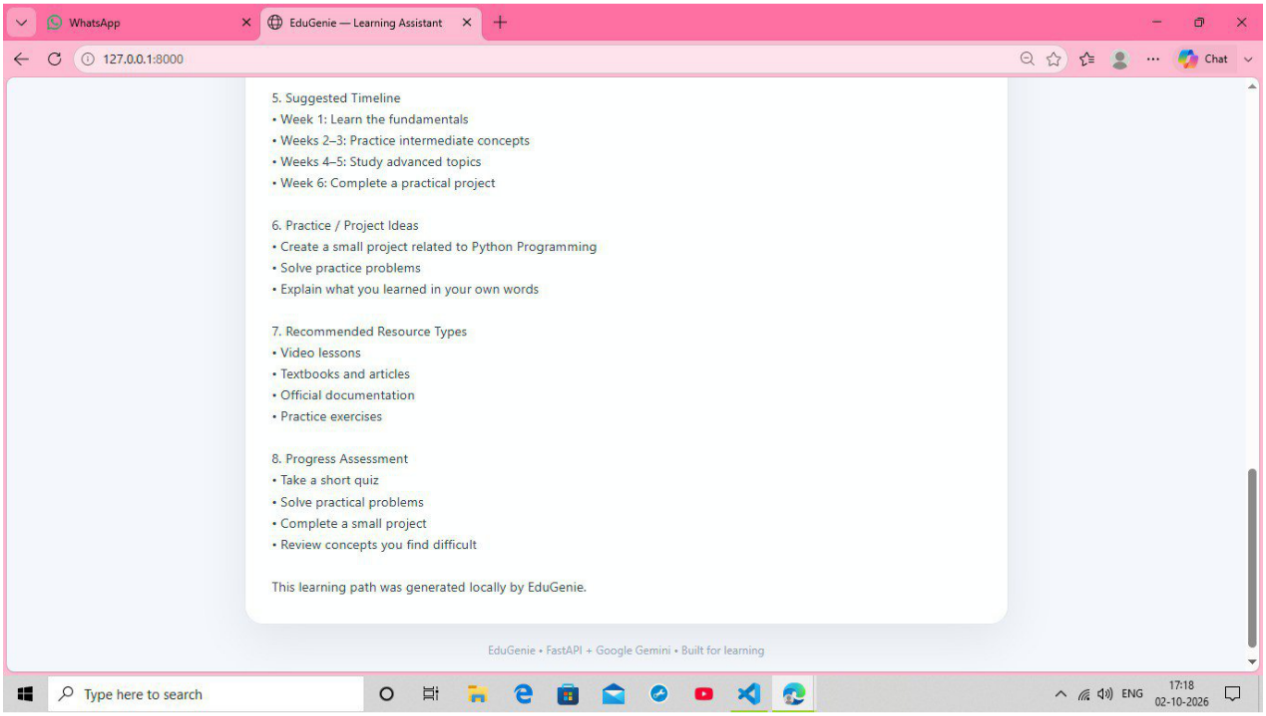


Figure 5.15: Learning Path output: suggested timeline, practice, resources and progress assessment.

Install Python 3.10 or later and verify with `python --version`.

Create and activate a virtual environment. Install the project's actual dependencies from requirements.txt.

Configure the Gemini API key securely as an environment variable or through a local .env file excluded from Git.

Install and configure the local LaMini-Flan-T5-783M model according to the actual implementation. The supplied document names the model but does not specify its exact loading library or download command.

5.2 Recommended repository structure

EduGenie/
■■■ main.py
■■■ explanation_module.py
■■■ qna.py
■■■ quiz_module.py
■■■ summary_module.py
■■■ learning_path.py
■■■ templates/
■ ■■■ index.html
■■■ static/
■ ■■■ style.css
■■■ requirements.txt
■■■ .env.example
■■■ .gitignore
■■■ README.md

This structure follows the folder architecture in the supplied project document. Add tests, screenshots, and other files as the implementation requires.

5.3 Backend and module implementation

Create the FastAPI application in main.py.

Implement separate functions in the five feature modules so that model-specific logic is not duplicated in route handlers.

Connect /qa to qna.py; /explain to explanation_module.py; /quiz to quiz_module.py; /summarize to summary_module.py; and /learn/recommendations to learning_path.py.

Validate incoming values and return predictable responses. Catch API, model-loading, generation, and parsing exceptions and return useful error information without exposing secrets.

5.4 AI feature implementation notes

Explanation: use LaMini-Flan-T5-783M locally to produce clear, concise explanations suitable for beginners.

Q&A: send the user's question to Gemini 1.5 Pro via API and return the generated answer.

Quiz: prompt Gemini to produce three MCQs from the passage, each with four options and a correct answer. Clean any Markdown code fence before JSON parsing; validate the parsed structure.

Summary: ask Gemini to condense the supplied text while preserving the main ideas and context.

Learning path: ask Gemini to organize topics from beginner to advanced and include suggested resources such as videos, articles, or books. Include learner level if the implemented UI/API collects it.

5.5 Frontend implementation

Build templates/index.html with the task dropdown, text area, submit button, and result container.

Build static/style.css with responsive layout and clear styling for buttons, inputs, and generated results.

On submission, send a POST request to the endpoint matching the selected task. Display the returned content beneath the form and show a clear error if the request fails.

Use safe rendering practices for model-generated text; avoid injecting untrusted HTML directly into the page.

5.6 Integration sequence

Run and verify the FastAPI service before connecting the frontend.

Test each endpoint independently with representative input.

Connect the frontend form to each endpoint and confirm that the correct module is called.

Verify that responses are displayed in the result container and errors do not break subsequent requests.

Run the five complete user scenarios from the project document.

5.7 Local execution

Install dependencies using the team's requirements file, for example: `pip install -r requirements.txt`.

Start the service using the documented command: `uvicorn main:app --reload`.

Open `http://127.0.0.1:8000` in a browser.

These commands reflect the supplied project document; adjust only if the actual repository uses a different application entry point or setup.

5.8 Development evidence to add

[Insert screenshot: project folder structure.]

[Insert screenshot: application home page.]

[Insert screenshot: successful Q&A result.]

[Insert screenshot: concept explanation result.]

[Insert screenshot: generated quiz with options and answer feedback.]

[Insert screenshot: summary result.]

[Insert screenshot: learning recommendations result.]

[Insert screenshot: terminal showing successful Uvicorn startup; ensure no API key or secret is visible.]

5.9 Implementation log

PHASE 6: Project Testing

6.1 Testing objectives

5.1A Five-Input Functional Evidence Mapping

Input Area 1 – Q&A / Ask a Question: screenshots 1, 2

Input Area 2 – Explain a Concept: screenshots 3, 4, 5

Input Area 3 – Generate a Quiz: screenshots 6, 7, 8, 9, 10

Input Area 4 – Summarize Text: screenshots 11, 12

Input Area 5 – Recommend a Learning Path: screenshots 13, 14, 15

Verify that the five learning tasks can be submitted and return usable results; confirm that frontend-to-backend integration works; verify the documented quiz structure; and check that invalid inputs and service failures are handled.

6.2 Test environment

Application version / Git commit: [Enter].

Operating system and version: [Enter].

Python version: [Enter].

Browser and version: [Enter].

Dependency versions: [Record from requirements/environment].

Gemini model/API configuration: [Record actual configured model; do not include API key].

Local explanation model: LaMini-Flan-T5-783M (confirm actual installed revision/configuration).

6.3 Functional test cases

6.4 Quiz output validation

Check that exactly three question objects are returned.

Check that every question has exactly four options.

Check that the correct answer corresponds to one of the options.

Check that the response can be parsed as JSON after any Markdown code-fence cleanup.

Record any malformed outputs and the corrective action taken.

6.5 Non-functional checks

Usability: verify that task selection, input, submission, and result display are understandable.

Reliability: repeat each main workflow and record failures or inconsistent behavior.

Performance: record response time for each feature under the test conditions; do not claim performance targets without measurements.

Security: verify that credentials are not present in committed files, screenshots, logs, or demo recordings.

Compatibility: record the browsers and operating systems actually tested.

6.6 Defect log

6.7 Test summary and sign-off

Total test cases executed: [Enter].

Passed: [Enter]. Failed: [Enter]. Blocked / not run: [Enter].

Known limitations: [Enter].

Tester(s): [Enter]. Test date: [Enter].

Faculty review / comments: [Enter].

Do not mark the project as fully tested until actual results and supporting evidence have been added.

PHASE 7: Project Documentation

7.1 Abstract

EduGenie is a lightweight AI-powered learning assistant that brings together question answering, simplified concept explanation, quiz generation, summarization, and personalized learning recommendations. The documented design uses FastAPI for backend services and an HTML/CSS frontend. Gemini 1.5 Pro via API supports Q&A, quiz generation, summarization, and learning recommendations, while LaMini-Flan-T5-783M is used locally for concept explanation. The application is intended to make study support accessible through a simple browser-based interface.

7.2 Introduction

Students and independent learners use different resources to clarify doubts, revise content, test their understanding, and plan what to learn next. EduGenie aims to bring these activities into one interface. A learner selects a task, supplies a question, topic, or passage, and receives a generated response.

7.3 Objectives and scope

Objectives: answer learning questions; explain concepts in simplified language; generate three-question MCQ quizzes; summarize educational text; and recommend a structured learning path.

Scope: local web application with five feature modules, FastAPI endpoints, model integration, and responsive HTML/CSS frontend. Additional capabilities such as voice interaction, multilingual support, mobile app, progress dashboards, gamification, LMS integration, and image/PDF input are described in the source as future enhancements, not confirmed baseline features.

7.4 System requirements

Software prerequisites stated in the source: Python 3.10+, FastAPI, HTML/CSS, Google Gemini API key, Uvicorn, and Jinja2. The local LaMini-Flan-T5-783M model is also used for explanation.

Hardware: the source says the application works well on devices such as Mac M1, but does not provide minimum CPU, RAM, or storage requirements. Record the actual development/test machine in this section.

7.5 Installation and configuration

Install Python 3.10+ and verify the version.

Obtain a Gemini API key through Google AI Studio as described in the source project document.

Create a virtual environment and install packages from requirements.txt.

Set the API key through a secure environment variable or local .env file. Never commit the real key.

Ensure the local LaMini-Flan-T5-783M model and its runtime dependencies are available as required by the implementation.

Start the app with `uvicorn main:app --reload` and open `http://127.0.0.1:8000`.

7.6 Functional description

Q&A: user enters a question; the backend invokes Gemini 1.5 Pro and displays the answer.

Explain: user enters a concept; the local LaMini-Flan-T5-783M model generates a simplified explanation.

Quiz: user supplies a passage; Gemini generates three MCQs with four options and correct answers. The application cleans and parses JSON output.

Summary: user supplies long text; Gemini returns a concise summary retaining key information.

Recommend Path: user enters a topic; Gemini creates a structured beginner-to-advanced learning path with suggested resources.

7.7 API reference

The source document identifies these paths but does not specify request/response field names. Add the exact schemas from the implemented FastAPI code before final submission.

7.8 User guide

Open the application in a browser after starting the local server.

Choose the required task from the dropdown.

Enter the question, concept, passage, or topic in the text area.

Click Submit and wait for the generated result.

Read the result. For quiz generation, review the questions, options, and indicated correct answers.

If an error appears, check the input, server terminal, network connection, and Gemini API configuration.

7.9 Limitations and responsible use

AI-generated answers may contain errors, omissions, or outdated information. Learners should cross-check important academic facts with trusted sources or instructors.

Cloud-based features depend on API access, network availability, provider quotas, and model behavior.

Local explanation performance depends on the device and model runtime.

The source document does not specify user authentication, persistent progress storage, or formal data retention controls. Do not claim these features exist unless implemented.

Do not submit confidential or sensitive personal information.

7.10 Future enhancements

Voice-based interaction for hands-free learning.

Multilingual support to reach learners across language backgrounds.

Mobile application and offline capabilities.

Progress tracking dashboards and learning analytics.

Gamification through badges, learning streaks, and challenges.

Adaptive learning paths based on learner strengths and weaknesses.

Group study sessions, teacher/parent dashboards, and LMS integrations such as Moodle or Google Classroom.

Image and PDF input for doubt solving and summarization; real-time synchronization and smart notifications.

7.11 Conclusion

EduGenie combines five AI-supported learning tasks in a lightweight web application. Its modular architecture separates feature logic from API routing and presentation. The supplied project document identifies potential benefits for self-learners and educational institutions while acknowledging challenges around output accuracy, performance, personalization, and data privacy. Further improvements can extend accessibility, learner tracking, and integration with educational platforms.

7.12 References and project-specific records

Primary project specification: “EduGenie: Google Gemini Powered Learning Assistant” (document supplied for this submission).

Official documentation to record/consult during implementation: Python; FastAPI; Uvicorn; Jinja2; Google AI for Developers / Gemini API. Add the exact URLs and access dates used by the team.

Repository URL: [Insert public GitHub URL]. Demo video URL: [Insert Google Drive Anyone-with-the-link URL].

PHASE 8: Project Demonstration

8.1 Demonstration objective

Present EduGenie, explain the learning problem it addresses, demonstrate the five documented capabilities, show the execution process, and conclude with the observed output. Record the complete demonstration with screen sharing and student voice-over, as required by the submission instructions.

8.2 Suggested demo duration and flow

8.3 Voice-over script

“Hello everyone. We are presenting our project, EduGenie: Google Gemini Powered Learning Assistant. EduGenie is designed to make common learning activities easier through one simple web application.”

“The application provides five features: question answering, simplified concept explanation, quiz generation, summarization, and personalized learning recommendations.”

“The backend is developed using FastAPI and the frontend uses HTML and CSS. As described in our project design, Gemini 1.5 Pro is used through an API for Q&A, quiz generation, summarization, and learning recommendations. LaMini-Flan-T5-783M is used locally for concept explanation.”

“First, we will demonstrate question answering. We select Q&A, enter a sample question, submit it, and review the response.”

“Next, we demonstrate concept explanation by entering a topic such as the Pythagoras Theorem. EduGenie returns a simplified explanation.”

“Now we demonstrate quiz generation. We provide an educational passage. The application generates three multiple-choice questions, each with four options and a correct answer.”

“For summarization, we enter a longer passage and review the concise version returned by the application.”

“Finally, we request a learning path for a topic such as SQL. The result organizes learning from beginner to advanced and suggests resources.”

“These demonstrations show how EduGenie brings several study-support tasks together. AI-generated content should still be checked against reliable learning materials. Thank you.”

8.4 Live demonstration checklist

Confirm the application starts successfully and the correct version is running.

Confirm the browser is open to the application home page.

Prepare tested sample inputs for all five features.

Check network connectivity and Gemini API availability before recording.

Ensure the API key is not visible in the terminal, editor, browser, or recording.

Show the project name and explain its purpose and benefits.

Demonstrate execution and output for all five features.

Speak clearly and explain what is happening while screen sharing.

Show the final output and close with a concise summary.

8.5 Suggested sample inputs

Q&A: “Which is the largest ocean?”

Explain: “Explain the Pythagoras Theorem in simple terms.”

Quiz: paste a short, factual educational passage that has been checked by the team.

Summary: paste a longer educational paragraph or passage.

Learning path: “Create a beginner-to-advanced learning path for SQL.”

8.6 Recording and submission requirements

Record the complete demonstration with screen sharing and the student's voice-over/explanation.

Include project name, purpose, use/benefits, execution/working process, and final output.

Upload the video to Google Drive and set general access to “Anyone with the link” with viewer permission.

Paste the shareable video URL here: [Insert Google Drive link].

Paste the public GitHub repository URL here: [Insert GitHub URL].

Before submission, test both links in a private/incognito browser window or while signed out.

8.7 Final presentation close

“EduGenie demonstrates how AI can support question answering, concept understanding, practice, revision, and learning planning through a single interface. The current version focuses on the five implemented modules. Future work may include voice interaction, multilingual support, mobile access, learner progress tracking, and LMS integration.”

APPENDIX A – FIVE INPUT AREA SCREENSHOT EVIDENCE

All 15 supplied screenshots are reproduced below and grouped into the five application input areas.

Input Area 1 – Q&A; / Ask a Question

The learner enters a question and receives an AI-generated answer.

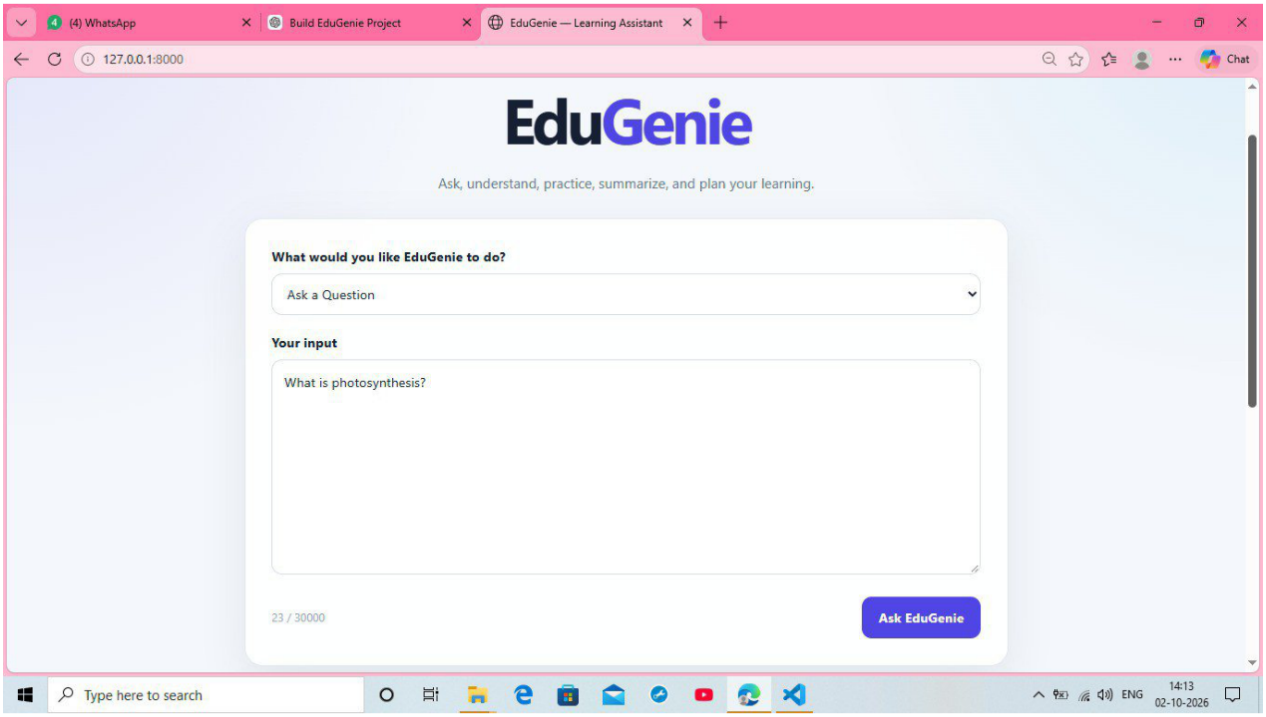


Figure A.1: Q&A; input: a photosynthesis question entered by the learner.

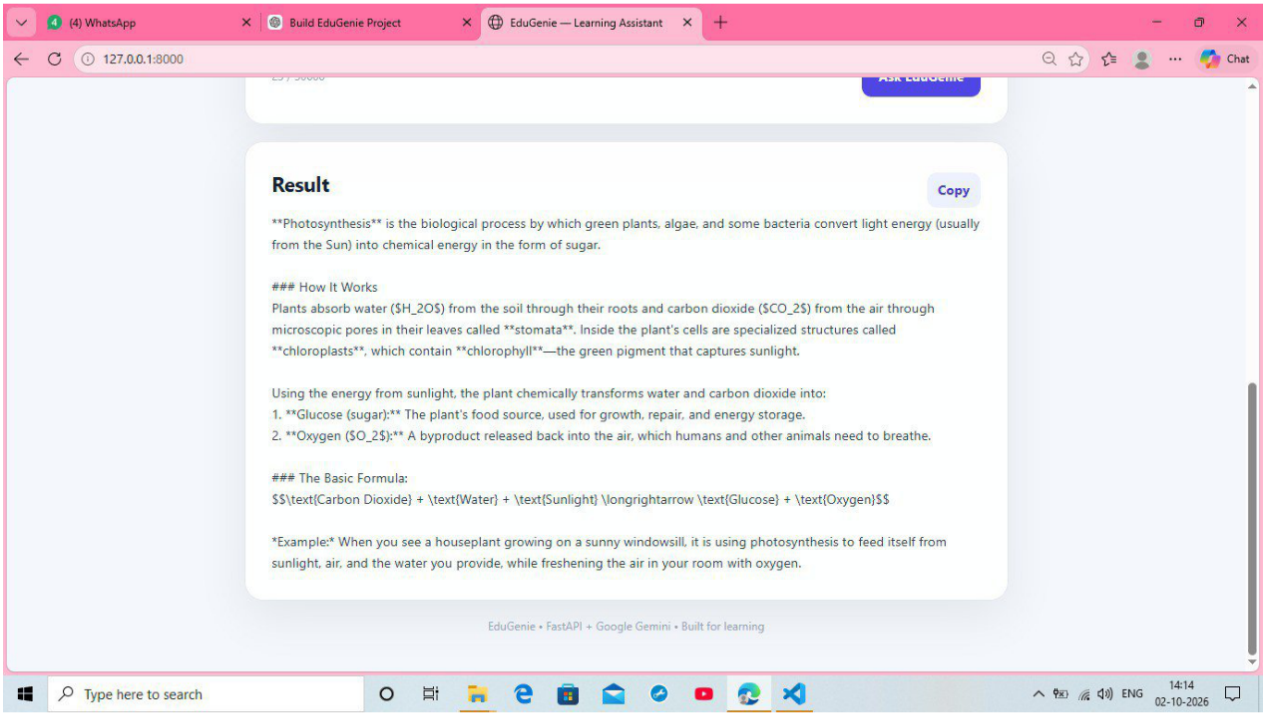


Figure A.2: Q&A; output: generated explanation of photosynthesis.

Input Area 2 – Explain a Concept

The learner enters a concept and receives a simplified, step-by-step explanation.

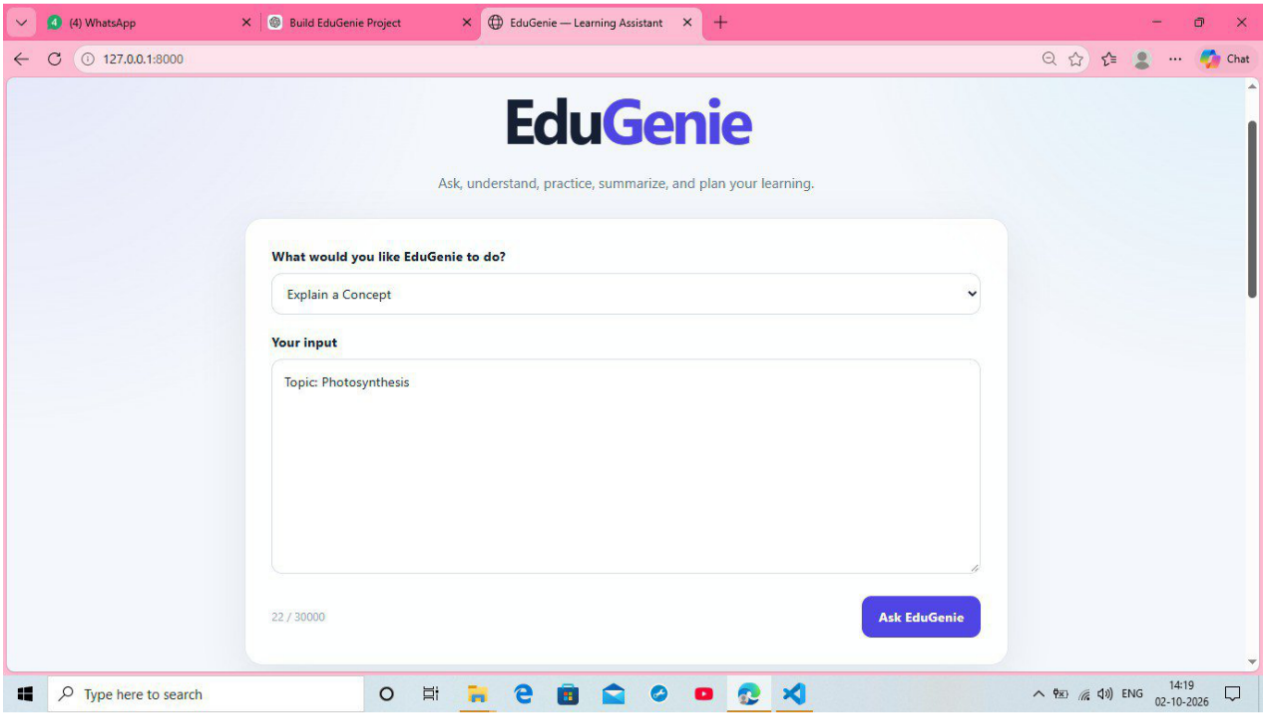


Figure A.3: Explain a Concept input: photosynthesis entered as the topic.

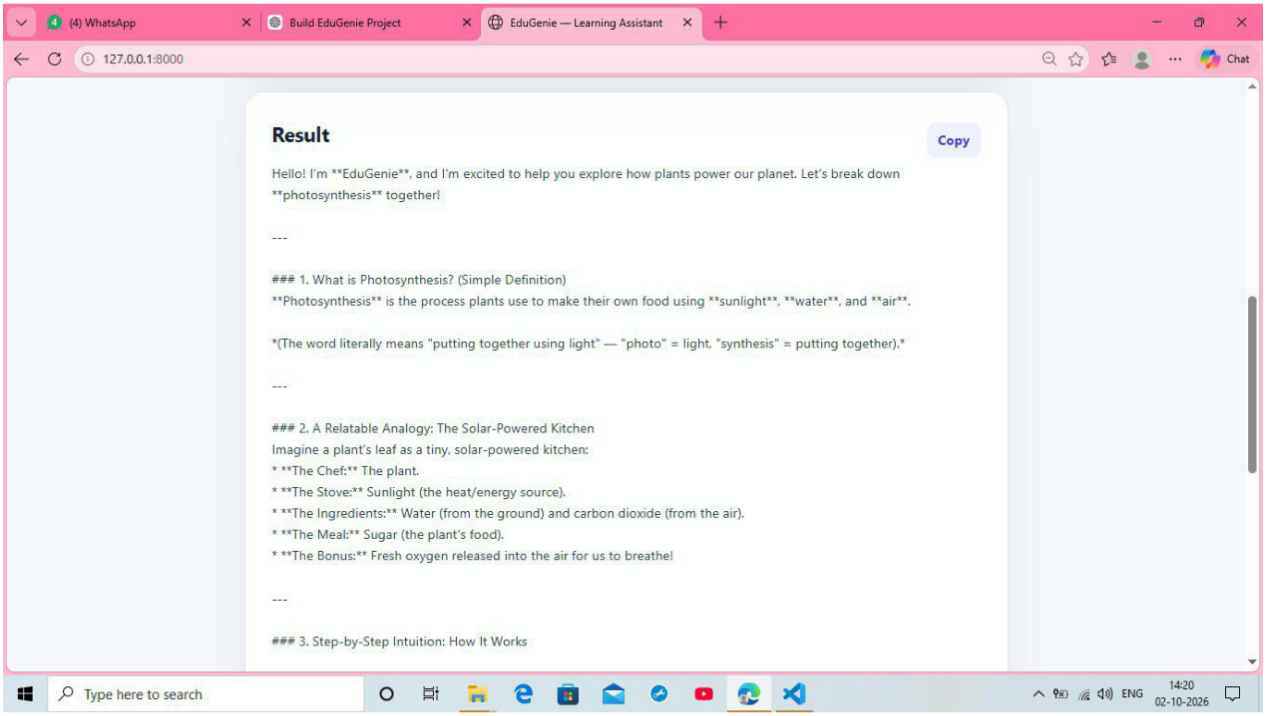


Figure A.4: Explain a Concept output: simplified explanation with analogy and key points.

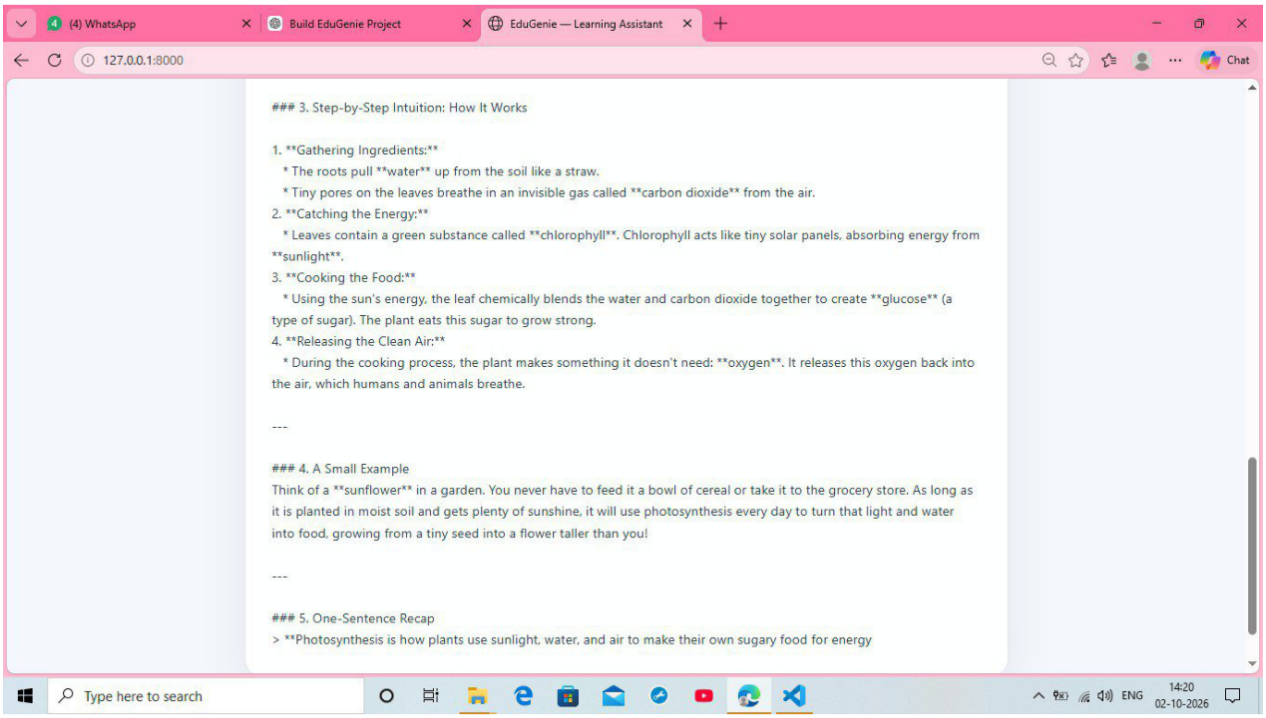


Figure A.5: Explain a Concept output: continued step-by-step explanation and recap.

Input Area 3 – Generate a Quiz

The learner enters a topic and receives multiple-choice quiz questions with answer choices and feedback.

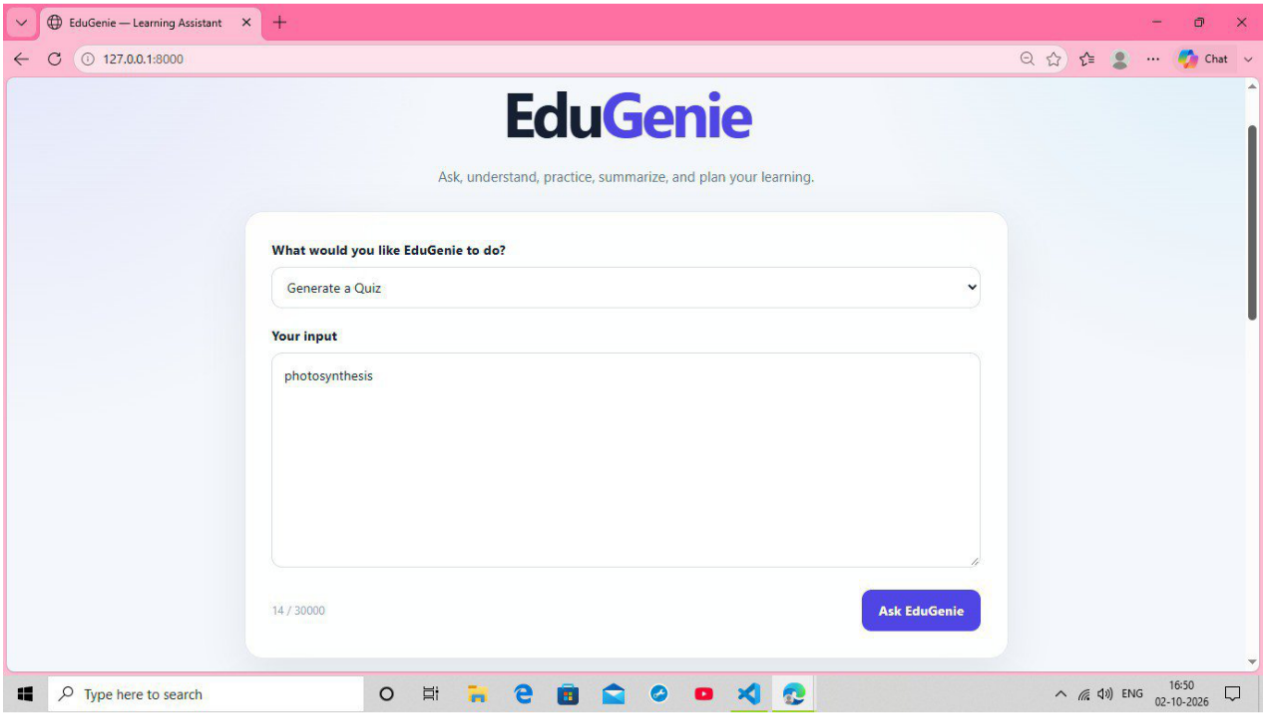


Figure A.6: Quiz input: photosynthesis entered as the quiz topic.

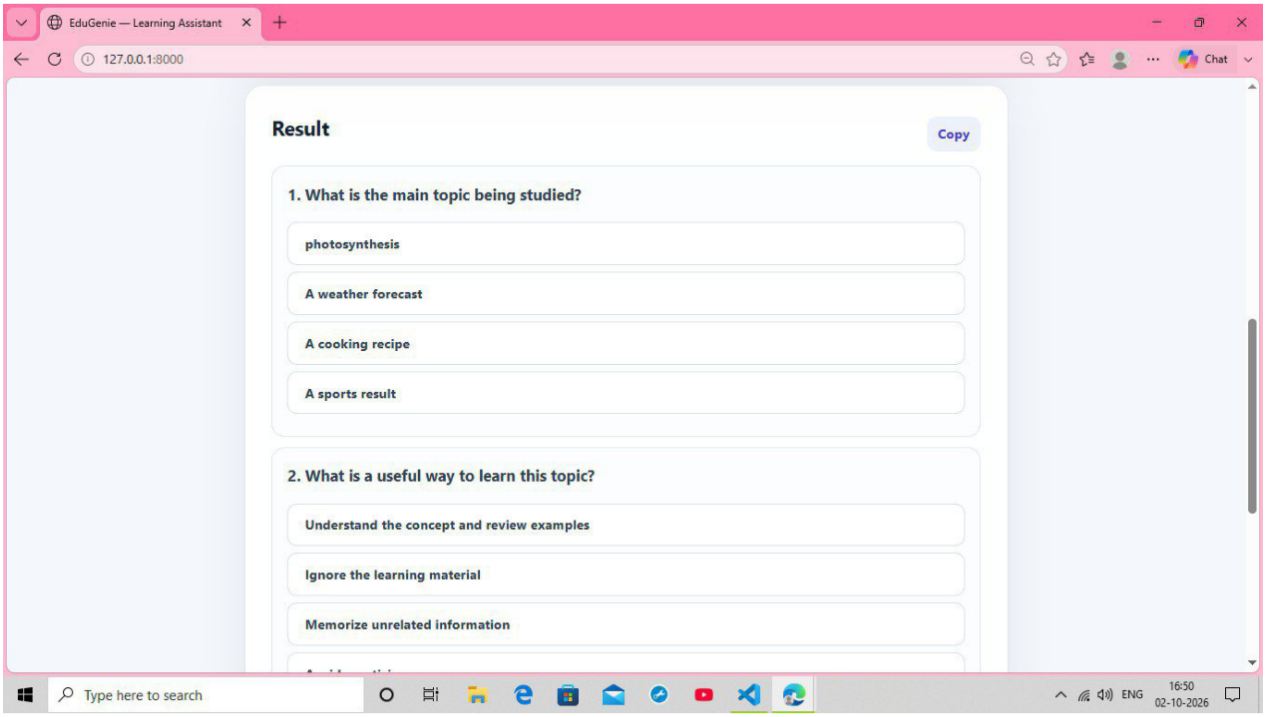


Figure A.7: Quiz output: generated multiple-choice questions and answer options.

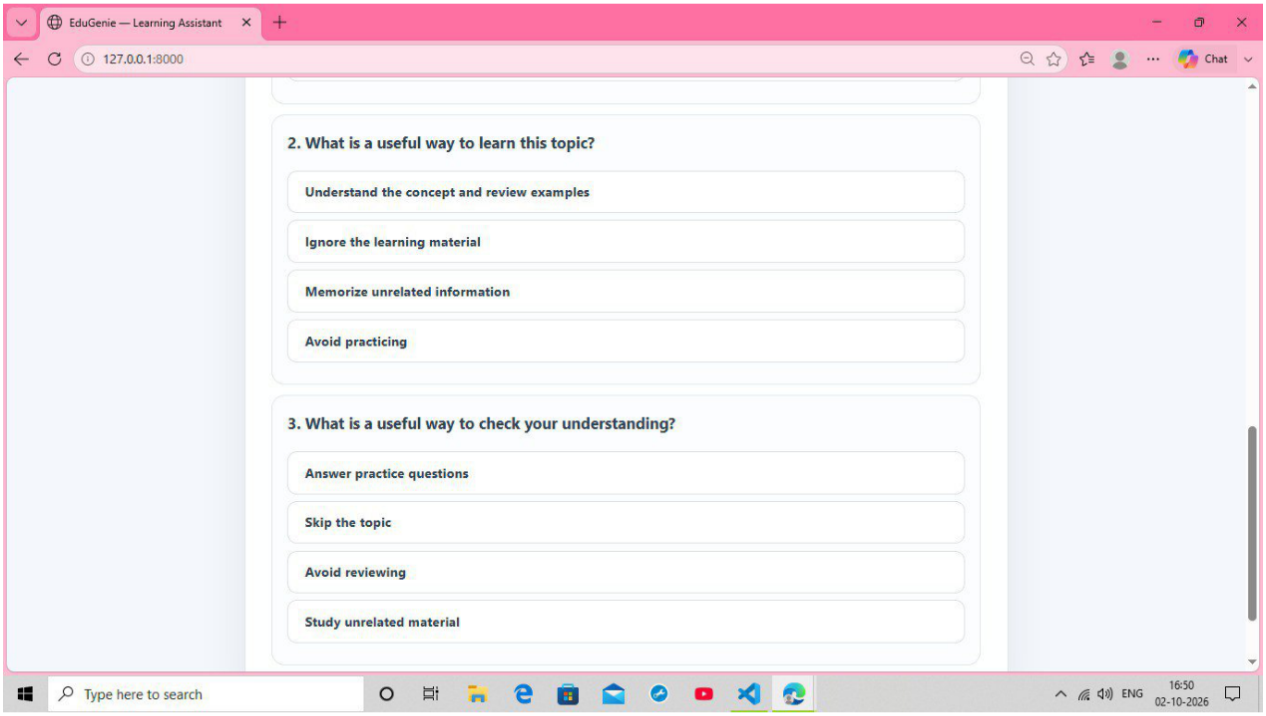


Figure A.8: Quiz output: additional multiple-choice questions and options.

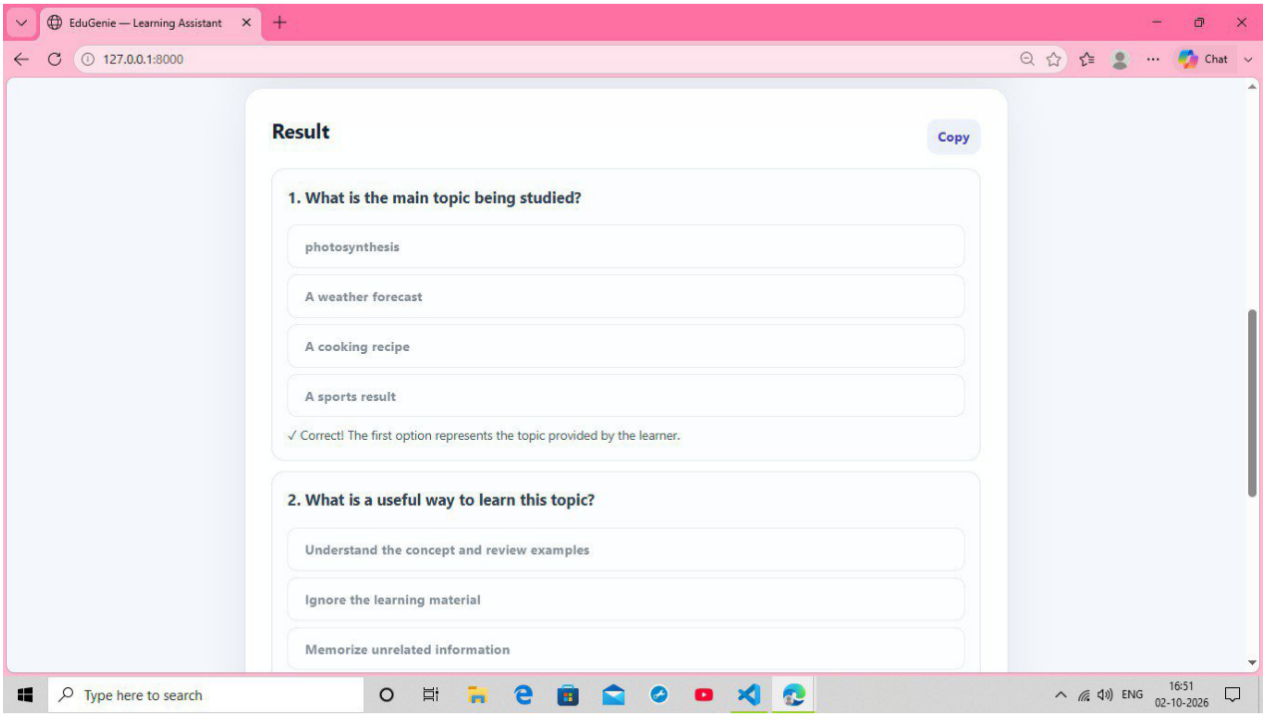


Figure A.9: Quiz output: answer selection and correctness feedback.

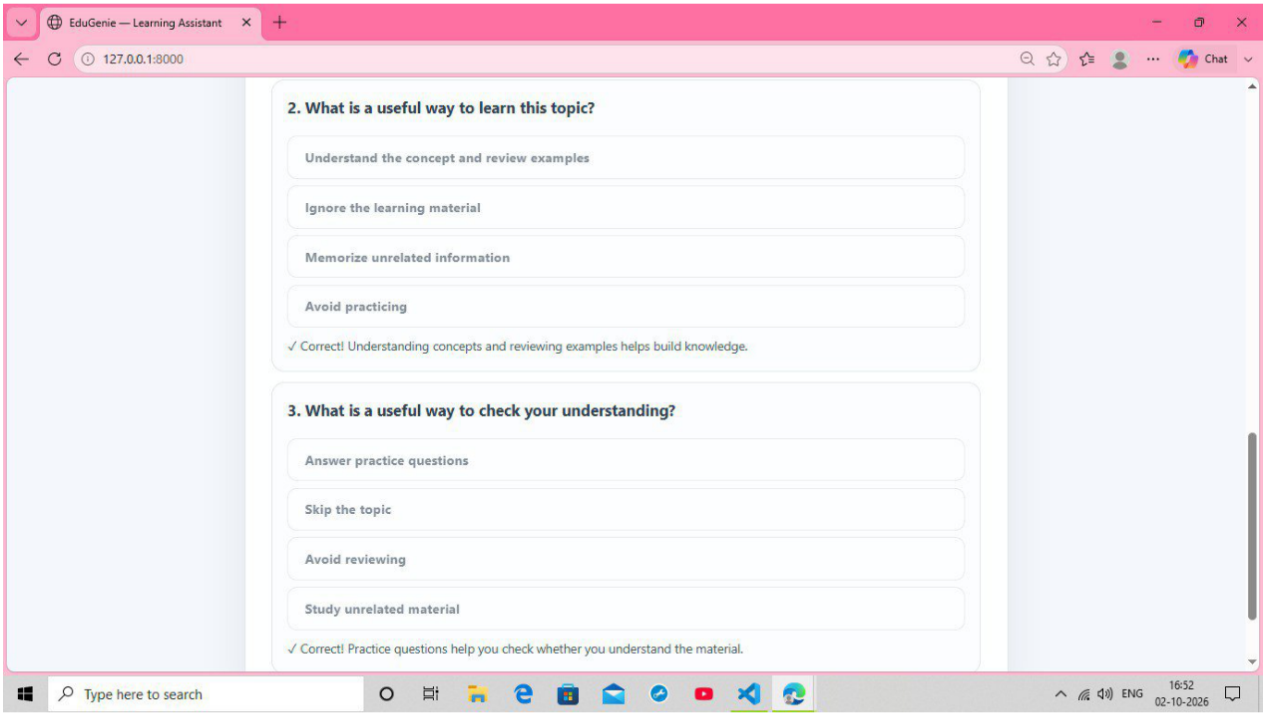


Figure A.10: Quiz output: further answer feedback and explanations.

Input Area 4 – Summarize Text

The learner supplies educational text and receives a concise summary.

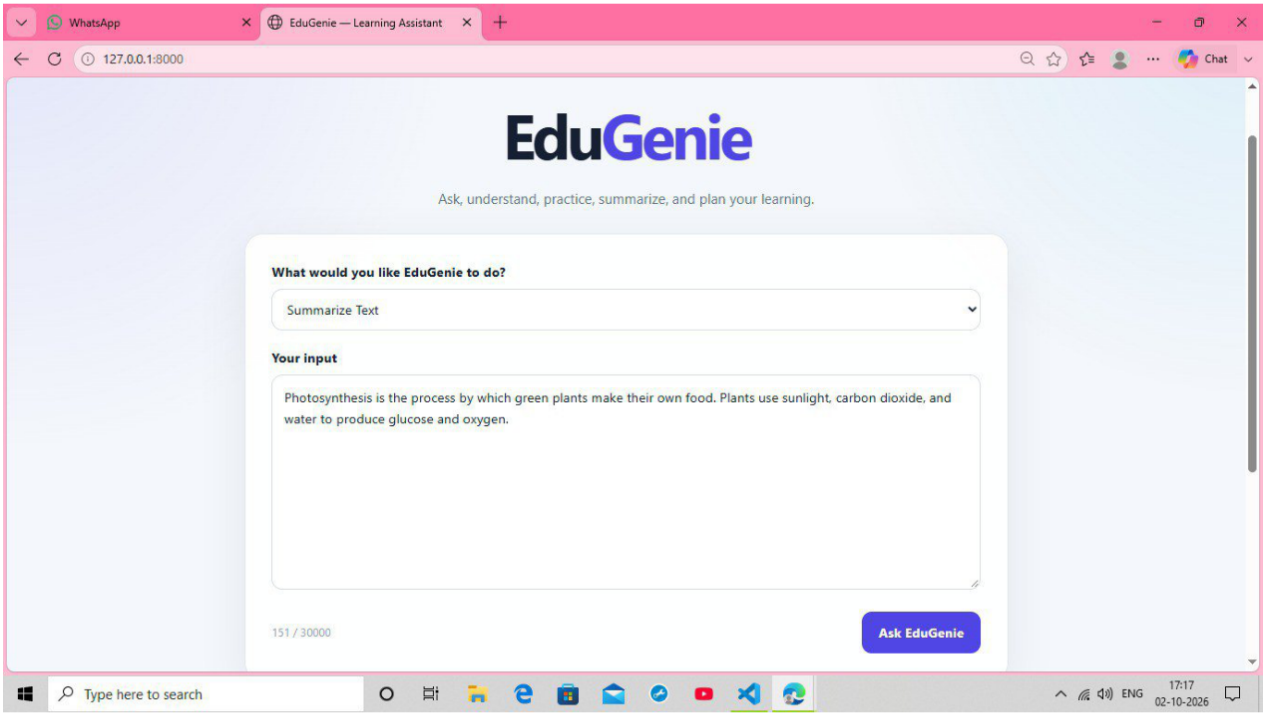


Figure A.11: Summary input: an educational photosynthesis passage supplied for summarization.

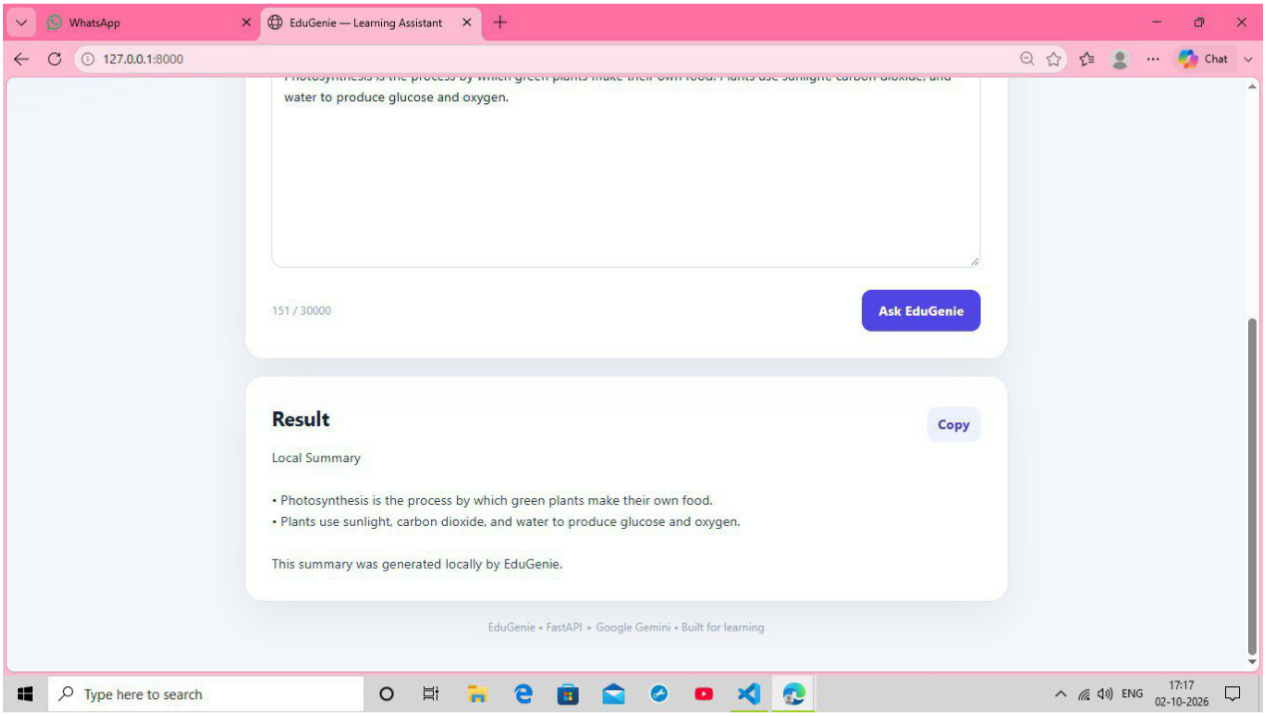


Figure A.12: Summary output: concise summary generated from the supplied passage.

Input Area 5 – Recommend a Learning Path

The learner supplies a subject and receives a structured learning path with stages, timeline, practice and assessment guidance.

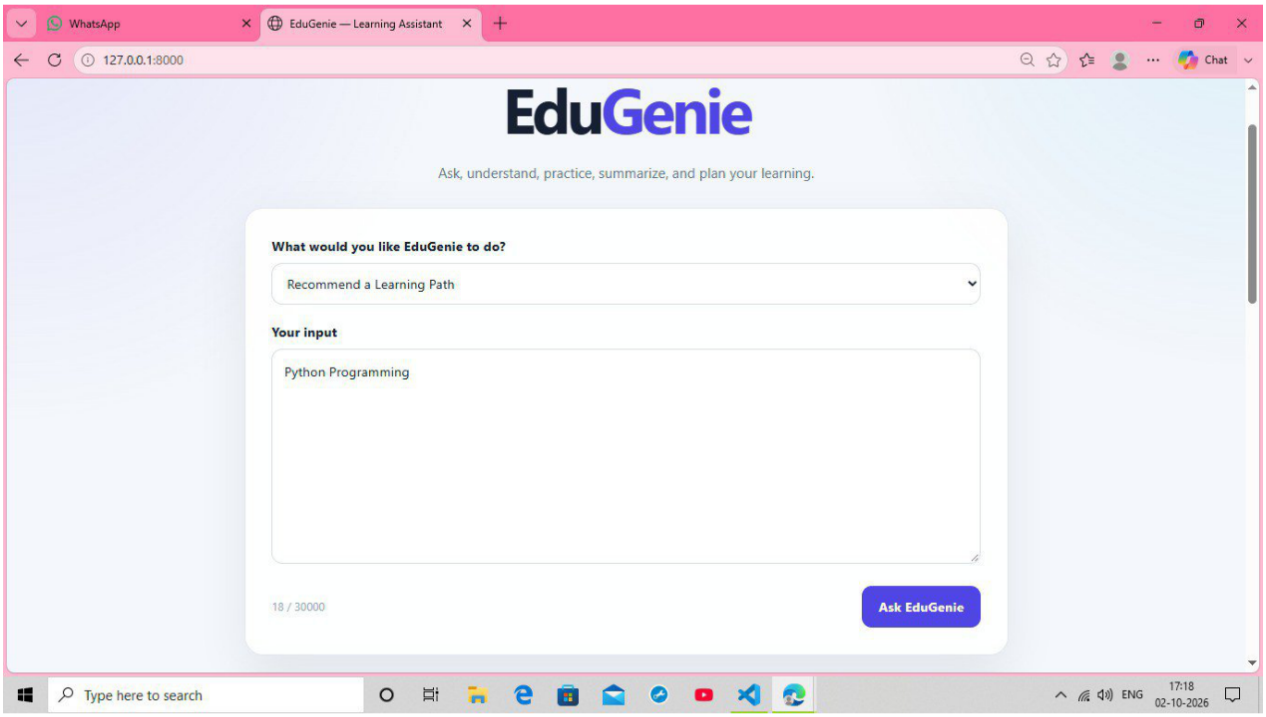


Figure A.13: Learning Path input: Python Programming entered as the learning subject.

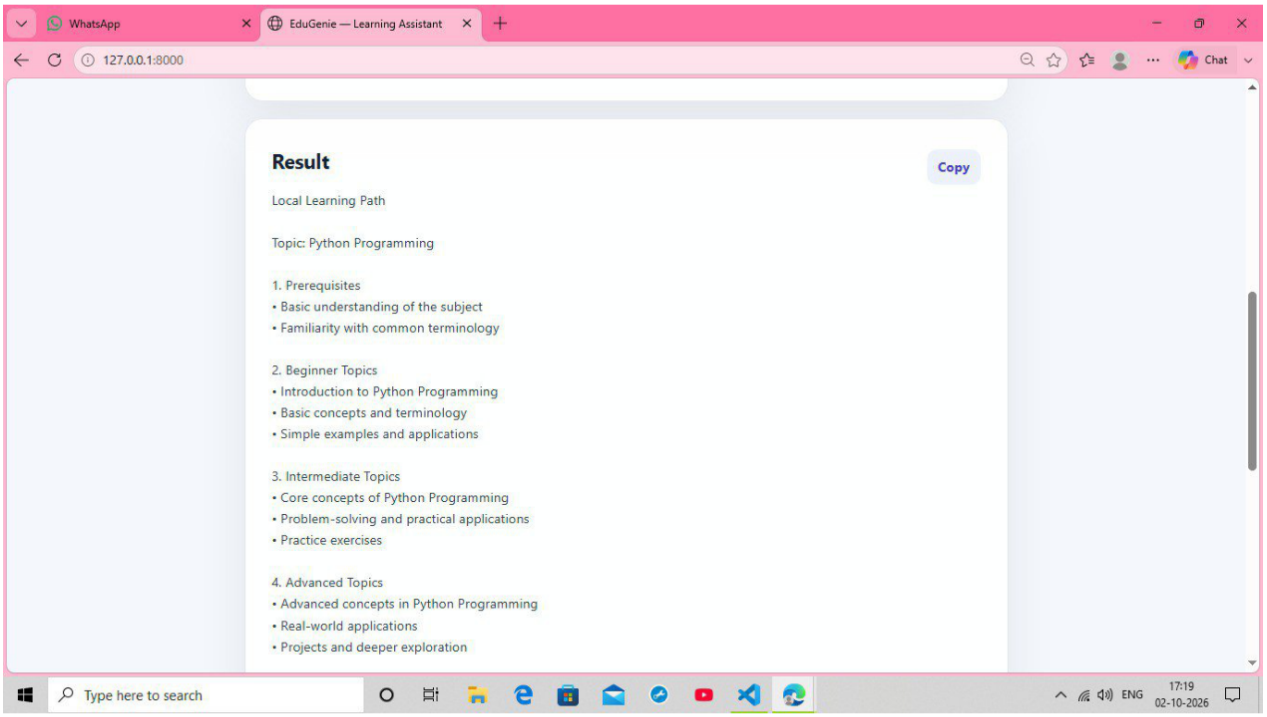


Figure A.14: Learning Path output: prerequisite, beginner, intermediate and advanced stages.

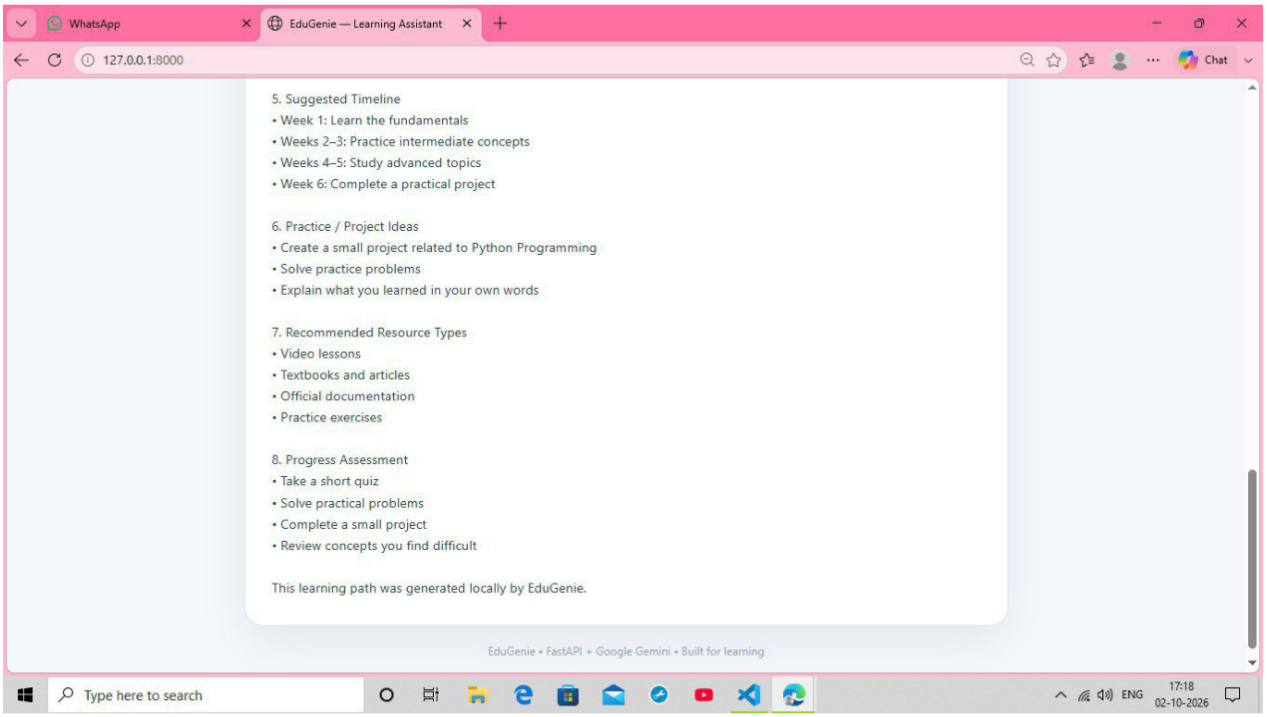


Figure A.15: Learning Path output: suggested timeline, practice, resources and progress assessment.